

# M07 — Hashing

---

**Sources:** CLRS Ch. 11 (Hash Tables) · Skiena §3.7 (Hashing), Ch. 6 §§6.2–6.7 (Bloom filters, perfect hashing, minwise hashing, Rabin–Karp)

---

## Big Idea

A hash table is a **direct-address table you can afford**: instead of allocating one slot per possible key, allocate  $\Theta(n)$  slots and compute the index from the key. That single move buys  $O(1)$  expected dictionary operations, and costs you two things — **collisions**, which need a resolution strategy (chaining or open addressing), and the **worst case**, which is  $\Theta(n)$  and cannot be argued away for any *fixed* hash function, because the pigeonhole principle hands an adversary a bad key set. The fix is the central idea of the module: **choose the hash function at random from a universal family**, which converts "there exists a bad input" into "there exist unlucky coin flips" — the same move as randomized quicksort. Beyond dictionaries, hashing is a general-purpose **many-to-one fingerprinting** tool: Bloom filters trade a one-sided error for a  $64\times$  space win, Rabin–Karp turns  $O(nm)$  string matching into expected  $O(n+m)$  with a rolling hash, minwise hashing estimates set similarity from  $k$  numbers, and canonicalization deliberately *engineers* collisions to group equivalent objects. Months later, remember:  $\alpha = n/m$  controls everything,  $1/(1-\alpha)$  is the open-addressing probe count, and collisions are a bug in a dictionary and a feature in a fingerprint.

---

## What You Should Be Able To Do After This Chapter

- Explain why direct addressing fails and exactly what a hash table trades for it.
- State the load factor  $\alpha$  and derive  $\Theta(1+\alpha)$  for chained search, both unsuccessful and successful.
- Define **universal** and  **$\epsilon$ -universal** families, and prove  $h_{\{ab\}}(k) = ((ak+b) \bmod p) \bmod m$  is universal.
- Implement multiply-shift hashing and say why  $a$  must be odd.
- Derive  $1/(1-\alpha)$  for unsuccessful open-addressing search and  $(1/\alpha) \ln(1/(1-\alpha))$  for successful, and quote the numbers at  $\alpha = 0.5$  and  $\alpha = 0.9$ .
- Explain why deletion needs tombstones under general open addressing — and

how linear probing avoids them.

- Explain **primary clustering**, and why it is a *liability* in the RAM model but an *asset* under a memory hierarchy.
  - Build a Bloom filter, derive the false-positive rate, and pick the optimal  $k$ .
  - Explain the two-level FKS perfect hashing scheme and why  $\sum \ell_i^2 = \Theta(n)$  is the condition that makes it linear-space.
  - Implement Rabin–Karp with a correct rolling hash.
  - Explain minwise hashing and why  $\Pr[\text{minhash collision}] = \text{Jaccard similarity}$ .
  - Recognize canonicalization and fingerprinting as *deliberate* uses of collisions.
- 

## 1. From direct addressing to hashing

### Direct-address tables [CLRS §11.1]

If keys are drawn from a small universe  $U = \{0, 1, \dots, m-1\}$ , just use an array  $T[0..m-1]$  where slot  $k$  holds the element with key  $k$  (or  $\text{NIL}$ ).

```
DIRECT-ADDRESS-SEARCH(T, k): return T[k]
DIRECT-ADDRESS-INSERT(T, x): T[x.key] = x
DIRECT-ADDRESS-DELETE(T, x): T[x.key] = NIL
```

→ **C++ implementation:** [A1 DIRECT-ADDRESS-SEARCH / INSERT / DELETE](#)

All  $O(1)$  **worst case**. CLRS's nice observation:

*Rather than storing an element's key and satellite data in an object external to the table, save space by storing the object directly in the slot. ... Then again, **why store the key of the object at all? The index of the object is its key!***

(A **bit vector** is the extreme case — one bit per key, no satellite data [Ex. 11.1-2].)

### The problem, and the move

If the universe  $U$  is large or infinite, storing a table  $T$  of size  $|U|$  may be impractical or even impossible. Furthermore, the set  $K$  of keys actually stored may be so small relative to  $U$  that **most of the space allocated for  $T$  would be wasted**. [CLRS §11.2, p.275]

**The hash table:** storage drops to  $\Theta(|K|)$  while search stays  $O(1)$ .

The catch is that **this bound is for the average-case time**, whereas for direct addressing it holds for the worst-case time.

A **hash function**  $h : U \rightarrow \{0, 1, \dots, m-1\}$  maps keys to slots; the element with key  $k$  goes to slot  $h(k)$ . Two keys mapping to the same slot is a **collision**.

**Collisions are unavoidable.** Since  $|U| > m$ , at least two keys must share a hash value. So we always need *both* a good hash function *and* a resolution strategy.

Of course, a hash function  $h$  must be **deterministic** in that a given input  $k$  must always produce the same output  $h(k)$ .

(Skiena's roulette-wheel picture [§3.7, p.93]: mapping a string to a huge integer and taking it  $\text{mod } m$  is like a ball travelling  $\lfloor H(S)/m \rfloor$  times around a circumference- $m$  wheel before settling into a bin — the long travel is what randomizes the landing spot.)

## Independent uniform hashing — the ideal we analyze against

An "ideal" hashing function  $h$  would have, for each possible input  $k$ , an output  $h(k)$  that is an element randomly and independently chosen uniformly from  $\{0, \dots, m-1\}$ . Once a value  $h(k)$  is randomly chosen, each subsequent call with the same input yields the same output. We call such an ideal hash function an **independent uniform hash function**. Such a function is also often called a **random oracle**. [CLRS §11.2, p.276]

Independent uniform hashing is **an ideal theoretical abstraction, but it is not something that can reasonably be implemented in practice**.

Two properties are all the analysis actually needs:

- **Uniformity** — each key is equally likely to hash to each of the  $m$  slots.
- **Independence** — where one key hashes doesn't affect where others do, so any two distinct keys collide with probability  $1/m$ .

That second property has a name: **universality**, and §3 shows it is achievable.

---

## 2. Collision resolution by chaining

### The structure

Slot  $j$  holds a linked list of every element whose hash value is  $j$ .

*At a high level, you can think of hashing with chaining as **a nonrecursive form of divide-and-conquer**: the input set of  $n$  elements is divided randomly into  $m$  subsets, each of approximate size  $n/m$ . [CLRS §11.2, p.277]*

```
CHAINED-HASH-INSERT(T, x): LIST-PREPEND(T[h(x.key)], x)
CHAINED-HASH-SEARCH(T, k): return LIST-SEARCH(T[h(k)], k)
CHAINED-HASH-DELETE(T, x): LIST-DELETE(T[h(x.key)], x)
```

→ **C++ implementation:** [A2 CHAINED-HASH-INSERT / SEARCH / DELETE](#)

| OPERATION | TIME                                                       | NOTE                                                            |
|-----------|------------------------------------------------------------|-----------------------------------------------------------------|
| INSERT    | $O(1)$ <b>worst case</b>                                   | Assumes $x$ is not already present.<br>Checking costs a search. |
| SEARCH    | proportional to the chain length                           | analyzed below                                                  |
| DELETE    | $O(1)$ <b>worst case</b> if lists are <b>doubly linked</b> | given a pointer to the element, not a key                       |

*If the hash table supports deletion, then its **linked lists should be doubly linked** in order to delete an item quickly. If the lists were only singly linked, then deletion could take time proportional to the length of the list.*

(That is exactly the predecessor-pointer problem from [M06 §6](#).)

### The load factor

$\alpha = n/m$  — the average number of elements per chain

$\alpha$  can be  $< 1$ ,  $= 1$ , or  $> 1$  under chaining. **Everything in this section is expressed in  $\alpha$ .**

**The worst case is terrible and we accept it:**

*All  $n$  keys hash to the same slot, creating a list of length  $n$ . The worst-case time for searching is thus  $\Theta(n)$  ... no better than using one linked list for all the elements. **We clearly don't use hash tables for their worst-case performance.***

## Theorem 11.1 — unsuccessful search

*In a hash table with chaining, an unsuccessful search takes  $\Theta(1 + \alpha)$  time on average, under independent uniform hashing.*

**Proof.** Key  $k$  is equally likely to hash to any of the  $m$  slots. The search walks to the end of  $T[h(k)]$ , whose expected length is  $E[n_{h(k)}] = \alpha$ . Adding the  $O(1)$  to compute  $h(k)$  gives  $\Theta(1 + \alpha)$ . ■

## Theorem 11.2 — successful search

*A successful search also takes  $\Theta(1 + \alpha)$  on average.*

**Why it isn't obvious:** an unsuccessful search is equally likely to go to any slot; a successful search **cannot land on an empty slot**, and the longer a list is, the more likely the search is for one of its elements. So successful search is biased toward long lists — yet the answer is the same.

**Proof sketch** (indicator random variables, M04). New elements are prepended, so the elements before  $x$  in its list were inserted *after*  $x$ . Define

$$X_{ijq} = I\{\text{the search is for } x_i, h(k_i) = q, \text{ and } h(k_j) = q\}$$

Since  $\Pr\{\text{search is for } x_i\} = 1/n$  and  $\Pr\{h(k_i) = q\} = \Pr\{h(k_j) = q\} = 1/m$ , and these are independent,  $E[X_{ijq}] = 1/nm^2$ . Summing over all slots  $q$  and pairs  $i < j$ , then applying linearity of expectation and  $\sum_{j=1}^n (j-1) = n(n-1)/2$ :

$$E[Z + 1] = 1 + m \cdot (n(n-1)/2) \cdot (1/nm^2) = 1 + (n-1)/2m = 1 + \alpha/2 - \alpha/2n$$

giving  $\Theta(2 + \alpha/2 - \alpha/2n) = \Theta(1 + \alpha)$ . ■

## The payoff

*If the number of elements is at most proportional to the number of slots,  $n = O(m)$ , and consequently  $\alpha = O(1)$ . Thus **searching takes constant time on average**. Since insertion takes  $O(1)$  worst case and deletion takes  $O(1)$  worst case with doubly linked lists, **we can support all dictionary operations in  $O(1)$  time on average**.*

And note what the proof actually depended on:

*The analysis depends only on **two essential properties**: uniformity, and independence (so any two distinct keys collide with probability  $1/m$ ).*

That is the hook for universal hashing.

## Skiena's operation table [§3.7.1, p.95]

Chaining with doubly linked lists,  $n$  items in  $m$  slots:

| OPERATION               | EXPECTED   | WORST CASE |
|-------------------------|------------|------------|
| Search( $L, k$ )        | $O(n/m)$   | $O(n)$     |
| Insert( $L, x$ )        | $O(1)$     | $O(1)$     |
| Delete( $L, x$ )        | $O(1)$     | $O(1)$     |
| Successor / Predecessor | $O(n + m)$ | $O(n + m)$ |
| Minimum / Maximum       | $O(n + m)$ | $O(n + m)$ |

**The  $O(n + m)$  entries are the important limitation.** A hash table destroys order entirely — every order-based query requires scanning **all  $m$  buckets**, even if  $n$  is tiny. (Compare [M08](#): a balanced BST gives all seven operations in  $O(\log n)$ .)

*Traversing all the elements takes  $O(n + m)$  time for chaining, since we have to scan all  $m$  buckets looking for elements, even if the actual number of inserted items is small. This reduces to  $O(m)$  for open addressing, since  $n$  must be at most  $m$ .*

Both books also note: initializing an  $m$ -slot table costs  $O(m)$ .

## C++ Implementation — chained hash table

```

#include <vector>
#include <list>
#include <utility>
#include <functional>
#include <cstdint>

// Dictionary via chaining. All operations O(1) expected under a
// good hash.
// Grows (rehashes) when the load factor exceeds maxLoad.
template <typename K, typename V, typename Hash = hash<K>>
class ChainedHashMap {
public:
    explicit ChainedHashMap(size_t buckets = 16, double maxLoad =
1.0)
        : table_(buckets), count_(0), maxLoad_(maxLoad) {}

    size_t size() const { return count_; }
    double loadFactor() const { return double(count_) /
table_.size(); }

    V* find(const K& key) {
        for (auto& kv : table_[slot(key)])
            if (kv.first == key) return &kv.second;
        return nullptr;
    }

    void insert(const K& key, const V& value) {
        if (V* p = find(key)) { *p = value; return; } //
overwrite
        if (loadFactor() >= maxLoad_) rehash(table_.size() * 2);
        table_[slot(key)].emplace_front(key, value); //
prepend: O(1)
        ++count_;
    }

    bool erase(const K& key) {
        auto& chain = table_[slot(key)];
        for (auto it = chain.begin(); it != chain.end(); ++it)
            if (it->first == key) { chain.erase(it); --count_;
return true; }
        return false;
    }
}

```

```

private:
    vector<list<pair<K, V>>> table_;
    size_t count_;
    double maxLoad_;
    Hash hash_;

    size_t slot(const K& key) const { return hash_(key) %
table_.size(); }

    void rehash(size_t newBuckets) {
        vector<list<pair<K, V>>> fresh(newBuckets);
        for (auto& chain : table_)
            for (auto& kv : chain)
                fresh[hash_(kv.first) %
newBuckets].push_front(move(kv));
        table_.swap(fresh);
    }
};

```

### Implementation notes.

- **Rehashing doubles**, so its amortized cost is  $O(1)$  per insert — the same geometric argument as dynamic arrays (M06 §3, M09).
- `insert` searches first, so the "assume not present" precondition of `CHAINED-HASH-INSERT` is discharged.
- `push_front` gives  $O(1)$  insertion, matching `LIST-PREPEND` — and it is what makes the successful-search proof's "elements before `x` were inserted after `x`" claim true.
- `std::unordered_map` is exactly this structure. Its **iterator stability** guarantee (only rehashing invalidates iterators) is why it must use chaining rather than open addressing.

## 3. Hash functions

What makes one good

*A good hash function satisfies (approximately) the assumption of independent uniform hashing. ... Unfortunately, you typically have no way to check this condition, unless you happen to know the probability distribution from which the keys are drawn. Moreover, the keys might not be drawn independently. [CLRS §11.3, p.283]*

Two families of approach:

| APPROACH              | IDEA                                                                  | VERDICT                                            |
|-----------------------|-----------------------------------------------------------------------|----------------------------------------------------|
| <b>Static hashing</b> | one fixed function that hopefully works on any data                   | <b>"no longer recommended"</b>                     |
| <b>Random hashing</b> | pick $h$ at random from a family, at runtime, independent of the data | <b>"We recommend that you use random hashing."</b> |

*This approach removes any need to know anything about the probability distribution of the input keys, as the randomization necessary for good average-case behavior then comes from the (known) random process used to pick the hash function, rather than from the (unknown) process used to create the input keys.*

## The mapping step [Skiena §3.7]

For a string  $s$  over an alphabet of size  $\alpha$ , treat the characters as digits in base  $\alpha$ :

$$H(s) = \sum_{i=0}^{|s|-1} \alpha^{|s|-(i+1)} \cdot \text{char}(s_i)$$

→ C++ implementation: [A4 The hash functions](#)

then reduce:  $H'(s) = H(s) \bmod m$ .

*If the table size is selected with enough finesse (ideally  $m$  is a large prime not too close to  $2^i - 1$ ), the resulting hash values should be fairly uniformly distributed.*

## Static method 1: division

$$h(k) = k \bmod m$$

→ C++ implementation: [A4 The hash functions](#)

Since it requires only a single division, hashing by division is quite fast. The division method *may work well when  $m$  is a prime not too close to an exact power of 2*. There is no guarantee that this method provides good average-case performance, however, and it may complicate applications since *it constrains the size of the hash tables to be prime*.

**The failure mode to know** [Ex. 11.3-3]: if  $m = 2^p - 1$  and  $k$  is a string in radix  $2^p$ , then **any permutation of the characters hashes to the same value** — disastrous for anagram-rich key sets.

## Static method 2: multiplication, and multiply-shift

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor \quad \text{for a constant } 0 < A < 1$$

The general multiplication method has the advantage that *the value of  $m$  is not critical* and you can choose it independently of  $A$ .

**The multiply-shift form** — the practical one. Let  $m = 2^\ell$ ,  $w$  = machine word size,  $a = A \cdot 2^w$  a fixed  $w$ -bit value:

$$h_a(k) = (ka \bmod 2^w) \gg (w - \ell) \quad \leftarrow \text{CLRS eq. (11.2)}$$

→ C++ implementation: [A4 The hash functions](#)

Multiplying two  $w$ -bit words gives  $2w$  bits; taking mod  $2^w$  keeps the **low** word  $r_0$ ; the right shift extracts the  $\ell$  **most significant bits of  $r_0$** . Three machine instructions: multiply, subtract (the shift amount), logical right shift.

**Worked example** [CLRS p.286]:  $k = 123456$ ,  $\ell = 14$ ,  $m = 16384$ ,  $w = 32$ ,  $a = 2654435769$  (Knuth's constant).  $ka = 327706022297664 = 76300 \cdot 2^{32} + 17612864$ , so  $r_0 = 17612864$ , and its top 14 bits give  **$h_a(k) = 67$** .

Even though the multiply-shift method is fast, it doesn't provide any guarantee of good average-case performance. ... A simple randomized variant works well on average, when the program begins by *picking  $a$  as a randomly chosen odd integer*.

## Random hashing and universal families [CLRS §11.3.2]

**The threat model, stated plainly:**

Suppose that a malicious adversary chooses the keys to be hashed by some fixed hash function. Then the adversary can choose  $n$  keys that all hash to the same slot, yielding an average retrieval time of  $\Theta(n)$ . **Any static hash function is vulnerable to such terrible worst-case behavior.** The only effective way to improve the situation is to **choose the hash function randomly in a way that is independent of the keys that are actually going to be stored.**

(Skiena's pigeonhole version of the same argument is in [MO4 §7](#): take any  $nm$  keys; some bucket must receive  $\geq n$  of them.)

**Definition.** A finite family  $\mathcal{H}$  of hash functions  $U \rightarrow \{0, \dots, m-1\}$  is **universal** if for each pair of distinct keys  $k_1, k_2 \in U$ , the number of  $h \in \mathcal{H}$  with  $h(k_1) = h(k_2)$  is at most  $|\mathcal{H}|/m$ . Equivalently: **a randomly chosen  $h \in \mathcal{H}$  collides any two distinct keys with probability  $\leq 1/m$**  — no worse than if the two hash values had been chosen independently at random.

## The property hierarchy [CLRS §11.3.3]

| PROPERTY                               | DEFINITION                                                                                                     |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>uniform</b>                         | for any key $k$ and slot $q$ , $\Pr\{h(k) = q\} = 1/m$                                                         |
| <b>universal</b>                       | for any distinct $k_1, k_2$ , $\Pr\{h(k_1) = h(k_2)\} \leq 1/m$                                                |
| <b><math>\epsilon</math>-universal</b> | for any distinct $k_1, k_2$ , $\Pr\{h(k_1) = h(k_2)\} \leq \epsilon$ . (Universal = $1/m$ -universal.)         |
| <b><math>d</math>-independent</b>      | for any distinct $k_1, \dots, k_d$ and any slots $q_1, \dots, q_d$ , $\Pr\{h(k_i) = q_i \ \forall i\} = 1/m^d$ |

Independent uniform hashing is the same as picking a hash function uniformly at random from a family of  $m^n$  hash functions. **Every independent uniform family is universal, but the converse need not be true:** consider  $U = \{0, \dots, m-1\}$  with the identity function as the only family member — the probability that two distinct keys collide is zero, even though each key hashes to a fixed value.

## Corollary 11.3 — the payoff

Using universal hashing and chaining in an initially empty table with  $m$  slots, it takes  **$\Theta(s)$  expected time** to handle any sequence of  $s$  INSERT, SEARCH, and DELETE operations containing  $n = O(m)$  INSERTs.

The proof is just Theorem 11.2 with  $1/m$  replaced by  $\leq 1/m$ . The analysis depended only on collision probabilities, so a universal family suffices.

*It becomes impossible for an adversary to pick a sequence of operations that forces the worst-case running time.*

## Theorem 11.4 — the number-theoretic universal family

Choose a prime  $p$  with every key in  $[0, p-1]$ , and  $p > m$ . For  $a \in \mathbb{Z}_p^* = \{1, \dots, p-1\}$  and  $b \in \mathbb{Z}_p = \{0, \dots, p-1\}$ :

$$h_{\{ab\}}(k) = ((ak + b) \bmod p) \bmod m$$

$$H_{\{pm\}} = \{ h_{\{ab\}} : a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p \} \quad - p(p-1) \text{ functions}$$

→ C++ implementation: [A4 The hash functions](#)

Example:  $p = 17, m = 6$ :  $h_{\{3,4\}}(8) = ((3 \cdot 8 + 4) \bmod 17) \bmod 6 = (28 \bmod 17) \bmod 6 = 11 \bmod 6 = 5$ .

*This family is universal. [Theorem 11.4]*

**Proof skeleton — three steps, and the structure is worth knowing:**

1. **No collisions at the "mod  $p$  level."** Let  $r_1 = (ak_1 + b) \bmod p, r_2 = (ak_2 + b) \bmod p$ . Then  $r_1 - r_2 \equiv a(k_1 - k_2) \pmod{p}$ . Since  $p$  is prime and both  $a$  and  $(k_1 - k_2)$  are nonzero mod  $p$ , their product is nonzero mod  $p$ . So  $r_1 \neq r_2$ .
2. **A bijection between  $(a, b)$  pairs and  $(r_1, r_2)$  pairs.** Given  $r_1 \neq r_2$  you can solve back:  $a = ((r_1 - r_2)((k_1 - k_2)^{-1} \bmod p)) \bmod p$  and  $b = (r_1 - ak_1) \bmod p$ . There are  $p(p-1)$  of each, so the correspondence is one-to-one. **Hence a uniformly random  $(a, b)$  makes  $(r_1, r_2)$  a uniformly random pair of distinct residues mod  $p$ .**
3. **Count the mod  $m$  collisions.** Fix  $r_1$ . Of the  $p-1$  other values for  $r_2$ , the number congruent to  $r_1 \bmod m$  is at most  $\lceil p/m \rceil - 1 \leq (p-1)/m$ . So the collision probability is at most  $((p-1)/m)/(p-1) = 1/m$ . ■

**Note the nice property:**  $m$  is arbitrary — it need not be prime, unlike in the division method.

## Theorem 11.5 — the practical family

We recommend that in practice you use the following hash-function family based on the multiply-shift method. It is exceptionally efficient and provably  $2/m$ -universal:

$$H = \{ h_a : a \text{ is odd, } 1 \leq a < m, h_a \text{ defined by equation (11.2)} \}$$

That is, the probability that any two distinct keys collide is at most  $2/m$ . In many practical situations, the speed of computing the hash function more than compensates for the higher upper bound on the collision probability.

**a must be odd** — that is what makes multiplication mod  $2^w$  a bijection.

## C++ Implementation — universal hash families

```
#include <stdint>
#include <random>
#include <string>
#include <vector>

// (1) Carter-Wegman universal family:  $h(k) = ((a*k + b) \bmod p) \bmod m$ .
//      Provably  $1/m$ -universal.  $p = 2^{61} - 1$  (a Mersenne prime).
class UniversalHash {
public:
    explicit UniversalHash(uint64_t m)
        : m_(m) {
        mt19937_64 rng(random_device{}());
        a_ = rng() % (kP - 1) + 1;      // a in [1, p-1]
        b_ = rng() % kP;               // b in [0, p-1]
    }
    uint64_t operator()(uint64_t k) const {
        const __uint128_t t = static_cast<__uint128_t>(a_) * (k %
kP) + b_;
        return static_cast<uint64_t>(t % kP) % m_;
    }
private:
    static constexpr uint64_t kP = (1ULL << 61) - 1;
    uint64_t a_, b_, m_;
};

// (2) Multiply-shift:  $h_a(k) = (k*a \bmod 2^{64}) \gg (64 - 1)$ .
//      Provably  $2/m$ -universal
```

```

//      for odd a and m = 2^l. Two instructions. Use this one in
practice.
class MultiplyShiftHash {
public:
    explicit MultiplyShiftHash(int l) : shift_(64 - l) {
        mt19937_64 rng(random_device{}());
        a_ = rng() | 1ULL;           // MUST be odd
    }
    uint64_t operator()(uint64_t k) const {
        return (k * a_) >> shift_;    // k*a_ wraps mod 2^64
        automatically
    }
private:
    uint64_t a_;
    int shift_;
};

// (3) Polynomial hash for variable-length input (CLRS Ex. 11.3-6):
//      h_b(<a0..a_{d-1}>) = (sum a_j * b^j) mod p, which is (d-
1)/p-universal.
//      This is also the Rabin-Karp hash (see section 9).
class StringHash {
public:
    StringHash() {
        mt19937_64 rng(random_device{}());
        b_ = rng() % (kP - 256) + 256; // base larger than the
        alphabet
    }
    uint64_t operator()(const string& s) const {
        uint64_t acc = 0;
        for (unsigned char c : s)
            acc = mulMod(acc, b_) + c, acc %= kP;    // Horner's
        rule
        return acc;
    }
private:
    static constexpr uint64_t kP = (1ULL << 61) - 1;
    uint64_t b_;
    static uint64_t mulMod(uint64_t x, uint64_t y) {
        return static_cast<uint64_t>(
            (static_cast<__uint128_t>(x) * y) % kP);
    }
}

```

```
};
```

### Implementation notes.

- $2^{61} - 1$  is prime and fits in `uint64`, so `__int128` products reduce without overflow. This is the standard competitive-programming modulus for hashing.
- `MultiplyShiftHash` relies on **unsigned overflow being defined** in C++ (wrapping mod  $2^{64}$ ). Signed overflow would be UB.
- `StringHash` uses **Horner's rule** (M01 mentions it), evaluating the polynomial in  $O(|s|)$  with no precomputed powers.
- Randomizing the base per process is exactly the **hash-flooding DoS mitigation** deployed in Python, Ruby, PHP, Node, and the Linux kernel.

## Long inputs: cryptographic hashing [CLRS §11.3.5]

*Cryptographic hash functions are complex pseudorandom functions, designed for applications requiring properties beyond those needed here, but are **robust, widely implemented, and usable as hash functions for hash tables**. For example, SHA-256 produces a 256-bit output for any input.*

To make a *family*, prepend a random **salt**:  $h_a(k) = \text{SHA-256}(a \parallel k) \bmod m$ .

*Cryptographic hash functions are useful because they provide a way of implementing **an approximate version of a random oracle**. From a theoretical point of view, a random oracle is an unachievable ideal. From a practical point of view, constructions based on cryptographic hash functions are **sensible substitutes**.*

**And the reason this is now practical** — CLRS's §11.5 argument, which is the modern engineering insight:

*Because of the memory hierarchy, **a complex computation that works entirely within the fast registers can take less time than a single read operation from main memory**.*

CLRS's own illustration is the "**wee**" hash function:  $f_a(k) = \text{swap}((2k^2 + ak) \bmod 2^w)$  iterated  $r = 4$  rounds, where `swap` exchanges the two  $w/2$ -bit halves. It is a tiny block cipher usable entirely in registers.

Experiments suggest that evaluating the wee hash function **takes less time than probing a single randomly chosen slot** in a hash table. ... For large hash tables, evaluating the wee hash function was **2 to 10 times faster than performing a single probe**.

## 4. Open addressing

### The structure [CLRS §11.4]

In open addressing, **all elements occupy the hash table itself**. Each table entry contains either an element or `NIL`. No lists or elements are stored outside the table. Thus the hash table can "**fill up**" so that no further insertions can be made. One consequence is that **the load factor  $\alpha$  can never exceed 1**.

The advantage of open addressing is that **it avoids pointers altogether**. Instead of following pointers, you compute the sequence of slots to be examined. The memory freed by not storing pointers provides the hash table with **a larger number of slots in the same amount of memory**, potentially yielding fewer collisions and faster retrieval.

The hash function takes a **probe number**:

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$$

**Requirement:** for every key `k`, the probe sequence  $\{h(k, 0), h(k, 1), \dots, h(k, m-1)\}$  must be a **permutation** of  $\{0, 1, \dots, m-1\}$ , so every slot is eventually considered.

| HASH-INSERT( <i>T</i> , <i>k</i> )                   | HASH-SEARCH( <i>T</i> , <i>k</i> )                           |
|------------------------------------------------------|--------------------------------------------------------------|
| 1 <i>i</i> = 0                                       | 1 <i>i</i> = 0                                               |
| 2 repeat                                             | 2 repeat                                                     |
| 3 <i>q</i> = <i>h</i> ( <i>k</i> , <i>i</i> )        | 3 <i>q</i> = <i>h</i> ( <i>k</i> , <i>i</i> )                |
| 4     if <i>T</i> [ <i>q</i> ] == NIL                | 4     if <i>T</i> [ <i>q</i> ] == <i>k</i> : return <i>q</i> |
| 5 <i>T</i> [ <i>q</i> ] = <i>k</i> ; return <i>q</i> | 6 <i>i</i> = <i>i</i> + 1                                    |
| 7     else <i>i</i> = <i>i</i> + 1                   | 7 until <i>T</i> [ <i>q</i> ] == NIL or <i>i</i> == <i>m</i> |
| 8 until <i>i</i> == <i>m</i>                         | 8 return NIL                                                 |
| 9 error "hash table overflow"                        |                                                              |

→ C++ implementation: [A3 HASH-INSERT / HASH-SEARCH \(open addressing\)](#)

**Why search may stop at an empty slot:** the search probes exactly the sequence insertion used, so  $k$  would have been placed at the first empty slot — if we reach an empty slot,  $k$  is not there.

## Deletion is the problem

*When you delete a key from slot  $q$ , it would be a mistake to mark that slot as empty by simply storing **NIL**. If you did, you might be unable to retrieve any key  $k$  for which slot  $q$  was probed and found occupied when  $k$  was inserted.*

**The standard fix: tombstones.** Store **DELETED** instead of **NIL**. **INSERT** treats it as empty; **SEARCH** passes over it.

### The cost:

*Using the special value **DELETED** means that search times no longer depend on the load factor  $\alpha$ , and for this reason chaining is frequently selected as a collision resolution technique when keys must be deleted.*

That is the honest trade-off: tombstones accumulate, and the probe sequence lengthens even as elements are removed. Only a rehash cleans them up.

## Probe sequences

**Linear probing** —  $h_2(k) = 1$  fixed:

$$h(k, i) = (h_1(k) + i) \bmod m$$

Only  $m$  distinct probe sequences (determined entirely by  $h_1(k)$ ).

**Double hashing** — the good general choice:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

*Double hashing offers one of the best methods available for open addressing because the permutations produced have many of the characteristics of randomly chosen permutations.*

**The critical constraint:**  $h_2(k)$  must be **relatively prime to  $m$** , or the probe sequence cycles through only  $1/d$  of the table where  $d = \gcd(m, h_2(k))$  [Ex. 11.4-5]. Two clean ways:

#### SCHEME HOW

$m = 2^\ell$  design  $h_2$  to always return an **odd** number

$m$  prime design  $h_2$  to return a positive integer  $< m$ ; e.g.  $h_1(k) = k \bmod m$ ,  $h_2(k) = 1 + (k \bmod m')$  with  $m' = m-1$

Double hashing produces  $\Theta(m^2)$  probe sequences — versus the  $m!$  of the ideal, but "as you might expect, seems to give good results."

## The analysis

Assume **independent uniform permutation hashing** (each key's probe sequence equally likely to be any of the  $m!$  permutations),  $\alpha < 1$ , and **no deletions**.

**Theorem 11.6 — unsuccessful search: at most  $1/(1-\alpha)$  probes.**

The intuition first:

*The bound  $1/(1-\alpha) = 1 + \alpha + \alpha^2 + \alpha^3 + \dots$  has an intuitive interpretation. **The first probe always occurs.** With probability approximately  $\alpha$ , the first probe finds an occupied slot, so a second probe happens. With probability approximately  $\alpha^2$ , the first two slots are occupied so a third probe ensues, and so on.*

The proof:  $\Pr\{X \geq i\} = (n/m)(n-1)/(m-1) \dots \leq \alpha^{i-1}$ , then  $E[X] = \sum \Pr\{X \geq i\} \leq \sum \alpha^{i-1} = 1/(1-\alpha)$ .

**Corollary 11.7.** Insertion takes at most  $1/(1-\alpha)$  probes — it is an unsuccessful search followed by a store.

**Theorem 11.8 — successful search: at most  $(1/\alpha) \cdot \ln(1/(1-\alpha))$  probes.**

The idea: a search for  $k$  reproduces  $k$ 's insertion probe sequence. If  $k$  was the  $(i+1)$ -st key inserted, the load factor then was  $i/m$ , so the expected probes are  $\leq m/(m-i)$ .

Average over all  $n$  keys and bound by an integral:

$$(1/n) \sum_{i=0}^{n-1} m/(m-i) = (1/\alpha) \sum_{k=m-n+1}^m 1/k \leq (1/\alpha) \int_{m-n}^m dx/x = (1/\alpha) \ln(1/(1-\alpha))$$

## The numbers to memorize

| A               | UNSUCCESSFUL $1/(1-A)$ | SUCCESSFUL $(1/A)\ln(1/(1-A))$       |
|-----------------|------------------------|--------------------------------------|
| 0.5             | 2                      | 1.387                                |
| 0.75            | 4                      | 1.848                                |
| 0.875           | 8                      | 2.377                                |
| 0.9             | 10                     | 2.559                                |
| $\rightarrow 1$ | $\rightarrow \infty$   | $H_m$ when $\alpha = 1$ [Ex. 11.4-4] |

**The asymmetry is the whole design lesson.** At 90% load a *successful* search still costs only 2.6 probes; an *unsuccessful* one costs 10. So the load factor is governed by your **miss** rate, not your hit rate — and this is why real implementations resize at  $\alpha$  between 0.5 and 0.75.

## C++ Implementation — open addressing with linear probing

```
#include <vector>
#include <cstdint>
#include <functional>
#include <optional>
#include <cstdint>

// Open-addressed hash map with linear probing and backward-shift
// deletion
// (no tombstones -- see CLRS section 11.5.1). Table size is a
// power of two.
template <typename K, typename V, typename Hash = hash<K>>
class LinearProbeMap {
public:
    explicit LinearProbeMap(size_t capacityPow2 = 16, double
maxLoad = 0.6)
        : slots_(capacityPow2), used_(capacityPow2, false),
          count_(0), maxLoad_(maxLoad) {}

    size_t size() const { return count_; }

    V* find(const K& key) {
        const size_t mask = slots_.size() - 1;
        for (size_t q = slot(key); used_[q]; q = (q + 1) & mask)
            if (slots_[q].first == key) return &slots_[q].second;
    }
};
```

```

        return nullptr; // hit an
empty slot -> absent
    }

    void insert(const K& key, const V& value) {
        if (V* p = find(key)) { *p = value; return; }
        if (double(count_ + 1) / slots_.size() > maxLoad_)
rehash(slots_.size() * 2);
        const size_t mask = slots_.size() - 1;
        size_t q = slot(key);
        while (used_[q]) q = (q + 1) & mask;
        slots_[q] = {key, value};
        used_[q] = true;
        ++count_;
    }

    // Deletion WITHOUT tombstones: after emptying slot q, walk
forward and pull
    // back any key whose ideal position is at or before q (CLRS
LINEAR-PROBING-
    // HASH-DELETE). Correct only for LINEAR probing.
    bool erase(const K& key) {
        const size_t mask = slots_.size() - 1;
        size_t q = slot(key);
        while (used_[q] && !(slots_[q].first == key)) q = (q + 1) &
mask;
        if (!used_[q]) return false;

        used_[q] = false;
        --count_;
        size_t j = q;
        while (true) {
            j = (j + 1) & mask;
            if (!used_[j]) return true; // run
ended

            const size_t ideal = slot(slots_[j].first);
            // Does the key at j need to move back into the hole at
q?

            // Equivalent to  $g(k', q) < g(k', j)$  with  $g(k, s) = (s - h_1(k)) \bmod m$ .
            if (((q - ideal) & mask) < ((j - ideal) & mask)) {
                slots_[q] = slots_[j];

```

```

        used_[q] = true;
        used_[j] = false;
        q = j;                                // the
hole moved to j
    }
}

private:
    vector<pair<K, V>> slots_;
    vector<bool> used_;
    size_t count_;
    double maxLoad_;
    Hash hash_;

    size_t slot(const K& key) const {
        return hash_(key) & (slots_.size() - 1);    // power-
of-two mask
    }

    void rehash(size_t newCap) {
        vector<pair<K, V>> oldSlots;
        oldSlots.reserve(count_);
        for (size_t i = 0; i < slots_.size(); ++i)
            if (used_[i]) oldSlots.push_back(slots_[i]);
        slots_.assign(newCap, {});
        used_.assign(newCap, false);
        count_ = 0;
        for (auto& kv : oldSlots) insert(kv.first, kv.second);
    }
};

```

## Implementation notes

- **Backward-shift deletion** implements CLRS's `LINEAR-PROBING-HASH-DELETE`. The inverse probe function is  $g(k, q) = (q - h_1(k)) \bmod m$  — "which probe number reaches slot  $q$ ". The test  $g(k', q) < g(k', q')$  asks *was the now-empty slot  $q$  probed before  $q'$  when  $k'$  was inserted?* If so,  $k'$  must move back or it becomes unreachable. **This works only for linear probing**, because every key follows the same cyclic sequence with a different start.
- **Power-of-two size + mask** replaces `%` with `&`. This requires a *good* hash

function — with a weak one, masking keeps only the low bits and clusters badly. (This is precisely why Java's `HashMap` XORs the high bits down before masking.)

- `maxLoad = 0.6` — comfortably inside the  $\alpha \leq 2/3$  bound of Theorem 11.9.
- `std::vector<bool>` is a bit-packed specialization; a good fit here.

## Common bugs

- Marking a deleted slot `NIL` under general open addressing  $\rightarrow$  keys become unreachable.
  - Double hashing where `h2(k)` shares a factor with `m`  $\rightarrow$  only `1/d` of the table is reachable.
  - `h2(k) = 0`  $\rightarrow$  infinite loop.
  - Masking with a power-of-two size and a weak hash  $\rightarrow$  severe clustering.
  - Forgetting that  $\alpha$  cannot exceed 1 — no overflow check.
- 

## 5. Practical considerations — linear probing and the memory hierarchy

[CLRS §11.5] — this section is what separates textbook hashing from production hashing.

### The two things the RAM model misses

**Memory hierarchies.** Each successive level stores more data but access is slower. ... **Cache memory is organized in cache blocks of (say) 64 bytes each, which are always fetched together.** There is a substantial benefit for ensuring that memory usage is local.

The standard RAM model measures efficiency by counting the number of slots probed. **In practice, this metric is only a crude approximation to the truth,** since once a cache block is in the cache, successive probes to that cache block are much faster.

**Advanced instruction sets.** Modern CPUs may have sophisticated instructions implementing primitives useful for encryption. **These may be useful in the design of exceptionally efficient hash functions.**

Primary clustering — a liability that becomes an asset

Linear probing exhibits a phenomenon known as **primary clustering**. Long runs of occupied slots build up, increasing the average search time. Clusters arise because **an empty slot preceded by  $i$  full slots gets filled next with probability  $(i+1)/m$** . Long runs tend to get longer.

### And then the reversal:

In the standard RAM model, primary clustering is a problem, and general double hashing usually performs better than linear probing. **By contrast, in a hierarchical memory model, primary clustering is a beneficial property**, as elements are often stored together in the same cache block. Searching proceeds through one cache block before advancing to search the next.

**Linear probing is often disparaged because of its poor performance in the standard RAM model. But linear probing excels for hierarchical memory models.**

### Theorem 11.9 — the guarantee

If  $h_1$  is **5-independent** and  $\alpha \leq 2/3$ , then it takes **expected constant time** to search for, insert, or delete a key in a hash table using linear probing.

(Indeed, the expected operation time is  $O(1/\epsilon^2)$  for  $\alpha = 1 - \epsilon$ .)

**The need for 5-independence is by no means obvious.**

### The design conclusion, and it is the modern one:

When the memory system is hierarchical, it becomes advantageous to use **linear probing**, since successive probes tend to stay in the same cache block. Furthermore, **hash functions that can be implemented using only the computer's fast registers are exceptionally efficient, so they can be quite complex and even cryptographically inspired**, providing the high degree of independence needed for linear probing to work most efficiently.

**In short: expensive hash function + linear probing + low load factor.** That is what Google's `absl::flat_hash_map`, Rust's `hashbrown/std::HashMap`, and Facebook's `F14` all do — and it is why they beat `std::unordered_map` (chained, iterator-stable, pointer-chasing) by 2–3× on typical workloads.

### The maximum-load caveat

From M04 §6.2 and [CLRS Problem 11-3]: throwing  $n$  keys into  $n$  slots gives a maximum chain length of  $\Theta(\lg n / \lg \lg n)$  in expectation. Skiena's own correction [§6.2]:

*Thus, I was a little too glib when I said in Section 3.7.1 that the worst-case access time for hashing is  $O(1)$ . ... the expected search time for hashing is  $O(1)$  averaged over all  $n$  keys, but we also expect there will be a few keys unlucky enough to require  $\Theta(\lg n / \log \log n)$  time.*

**Hash-table tail latency is not  $O(1)$ .** If your p99.9 matters, this is why.

---

## 6. Hashing beyond dictionaries

*I once heard Udi Manber — at one point responsible for all search products at Google — talk about the algorithms employed in industry. The three most important algorithms, he explained, were "**hashing, hashing, and hashing.**" [Skiena §3.7.2, p.95]*

*The key idea of hashing is to **represent a large object by a single number**. We get a representation of the large object by a value that can be manipulated in constant time, such that it is relatively unlikely that two different large objects map to the same value.*

### Three applications, each with a different structure

**1. Is a given document unique in a large corpus?** Comparing  $D$  against all  $n$  previous documents is hopeless. Hash  $D$  and compare  $H(D)$  against the corpus's hash codes; only on a collision do you compare explicitly. Since spurious collisions are rare, total effort is small.

**2. Is part of this document plagiarized?**

*This is a **more difficult** problem. Adding, deleting, or changing even one character completely changes the hash code. The hash codes from the previous application thus cannot help.*

*However, we could build a hash table of **all overlapping windows (substrings) of length  $w$**  in all documents. Whenever there is a match of hash codes, there is likely a common substring of length  $w$ . We should choose  $w$  long enough that such a co-occurrence is very unlikely by chance.*

The biggest downside is that **the size of the hash table becomes as large as the document corpus itself**. Retaining a small but well-chosen subset of these hash codes is exactly the goal of **min-wise hashing** (§8).

**3. How can I convince you that a file isn't changed?** Closed-bid auctions: everyone submits a **hash of their bid** before the deadline, then the full bid afterward. The auctioneer verifies the hash matches.

Such **cryptographic hashing** methods provide a way to ensure that the file you give me today is the same as the original, because any change to the file will change the hash code.

(That is a **commitment scheme**, and the same primitive underlies git object IDs, blockchain, and content-addressed storage.)

## Canonicalization — collisions on purpose

[Skiena §3.7.4, p.96]

**Problem.** Given letters  $s = (a, e, k, l)$ , find all dictionary words made by reordering them. Testing every word against  $s$  is linear in  $n$  per query.

**The trick: sort each word's letters** and hash on the result. `kale`, `lake`, `leak` all become `aekl` and land in the same bucket.

Once you have built this hash table, you can use it for different query sets  $s$ . The time for each query will be **proportional to the number of matching words**, which is a lot smaller than  $n$ .

**The bonus question.** "Which set of  $k$  letters can be used to make the most dictionary words?" This looks like it needs  $\alpha^k$  work. But:

Observe that the answer is simply **the hash code with the largest number of collisions**. Sweeping over a sorted array of hash codes makes this fast and easy.

This is a good example of the power of **canonicalization**, reducing complicated objects to a standard form. String transformations like lower-casing or **stemming** result in increased matches. **Soundex** is a canonicalization scheme for names, so spelling variants of "Skiena" — "Skina", "Skinnia", "Schiena" — all hash to **S25**.

**For hash tables, collisions are very bad. But for pattern matching problems like these, collisions are exactly what we want.**

**That inversion is the single most useful idea in this section.** Design your hash so that *equivalent* objects collide and *inequivalent* ones don't.

## Compaction / fingerprinting

[Skiena §3.7.5, p.97]

Sorting all books in a library by their full text is expensive — each comparison touches ~100,000 words. Instead, represent each book by its **first 100 characters**, sort those, and resolve the rare collisions by comparing full texts.

*The world's fastest sorting programs use this idea.*

*This is an example of **hashing for compaction**, also called **fingerprinting**, where we represent large objects by small hash codes. It is easier to work with small objects than large ones, and the hash code generally preserves the identity of each item. The hash function here is trivial (just take the prefix) but **it is designed to accomplish a specific goal — not to maintain a hash table.***

**Engineering examples:** git's SHA-1/SHA-256 object IDs, ETags, rsync's rolling checksums, content-addressed deduplication, Bloom filters (§7).

---

## 7. Bloom Filters

[Skiena §6.4, p.182]

### The setup

Return to the duplicate-detection problem. A hash table works, but you must store the documents to resolve collisions.

*But in this application, **spurious collisions are not really a tragedy**: they only mean that Google will fail to index a new document it has found. This can be an acceptable risk, provided the probability is low enough. **Removing the need to explicitly resolve collisions has big benefits in making the table smaller.** By reducing each bucket from a pointer link to a single bit, we **reduce the space by a factor of 64** on typical machines.*

### Why one bit per key isn't enough

With  $m$  bits set out of  $n$ , a new document falsely collides with probability  $p = m/n$ .

*Even if the table is only 5% full, there is still a  $p = 0.05$  probability that we will falsely discard a new discovery, which is much higher than is acceptable.*

## The Bloom filter

Hash each key with  $k$  **different hash functions**. Insert: set all  $k$  bits  $h_1(s), \dots, h_k(s)$ .

Query: return "present" only if **all  $k$  bits are 1**.

**The false-positive rate.** With  $m$  documents in an  $n$ -bit filter, at most  $km$  bits are set, so a single bit collides with probability  $p_1 = km/n$ . All  $k$  must collide:

$$p_k = (km/n)^k$$

→ C++ implementation: [A6 The Bloom filter](#)

*This is a peculiar expression, because a probability raised to the  $k$ -th power quickly becomes smaller with increasing  $k$ , yet here the probability being raised simultaneously increases with  $k$ . To find the  $k$  that minimizes  $p_k$ , we could take the derivative and set it to zero.*

**The trade-off curve** [Fig. 6.7]:

*Using a large number of hash functions reduces false-positive error substantially over a conventional hash table ( $k = 1$ ), at least for small loads. But the error rate associated with larger  $k$  increases rapidly with load, so for any given load there is always a point where adding more hash functions becomes counter-productive.*

**The headline number:**

*For a 5% load, the error rate for a simple hash table ( $k = 1$ ) will be 51.2 times larger than a Bloom filter with  $k = 5$  (error  $9.77 \times 10^{-4}$ ), even though they use exactly the same amount of memory.*

*A Bloom filter is an excellent data structure for maintaining an index, provided you can live with occasionally saying yes when the answer is no.*

**The critical property: one-sided error**

- "Not present"  $\Rightarrow$  **definitely** not present. (No false negatives.)
- "Present"  $\Rightarrow$  probably present. (False positives possible.)

This is a **Monte Carlo algorithm with one-sided error** (M04 §5) — the useful kind, because you can design around it: use the filter as a *cheap pre-filter* and confirm positives against the real store.

**And note:** you cannot delete from a standard Bloom filter (clearing a bit might unset it for another key). Counting Bloom filters replace bits with small counters to allow it.

## C++ Implementation

```
#include <vector>
#include <cstdint>
#include <cmath>
#include <string>
#include <cstdint>

// Bloom filter: bits_ is an n-bit array, k independent hash
// functions.
// insert: O(k).  mightContain: O(k).  No false negatives.
class BloomFilter {
public:
    // Size for an expected number of items and a target false-
    // positive rate.
    // Optimal:  n = -expected*ln(p) / (ln 2)^2  bits,  k =
    // (n/expected)*ln 2.
    BloomFilter(size_t expectedItems, double falsePositiveRate) {
        const double bitCount = -double(expectedItems) *
log(falsePositiveRate)
                                / (log(2.0) * log(2.0));
        nBits_ = max<size_t>(64, static_cast<size_t>(bitCount) +
1);
        hashCount_ = max<int>(1, static_cast<int>(
round(bitCount / double(expectedItems) *
log(2.0))));
        bits_.assign((nBits_ + 63) / 64, 0);
    }

    void insert(const string& seed) {
        auto [h1, h2] = baseHashes(seed);
```

```

        for (int i = 0; i < hashCount_; ++i) setBit(nth(h1, h2,
i));
    }

    // false => DEFINITELY absent.  true => probably present.
    bool mightContain(const string& seed) const {
        auto [h1, h2] = baseHashes(seed);
        for (int i = 0; i < hashCount_; ++i)
            if (!testBit(nth(h1, h2, i))) return false;
        return true;
    }

    int numHashes() const { return hashCount_; }
    size_t numBits() const { return nBits_; }

private:
    vector<uint64_t> bits_;
    size_t nBits_ = 0;
    int hashCount_ = 0;

    // Kirsch-Mitzenmacher: k hashes from 2 via g_i(x) = h1 + i*h2.
    // Provably as good asymptotically as k independent hashes.
    size_t nth(uint64_t h1, uint64_t h2, int i) const {
        return static_cast<size_t>((h1 + uint64_t(i) * h2) %
nBits_);
    }

    static pair<uint64_t, uint64_t> baseHashes(const string& seed)
{
    uint64_t a = 1469598103934665603ULL;        // FNV-1a
    for (unsigned char c : seed) { a ^= c; a *=
1099511628211ULL; }
    uint64_t b = 14695981039346656037ULL;        // a second,
different seed
    for (unsigned char c : seed) { b = (b * 31) + c; }
    return {a, b | 1ULL};                        // keep h2
nonzero
}

    void setBit(size_t i) { bits_[i >> 6] |= (1ULL << (i &
63)); }

    bool testBit(size_t i) const { return (bits_[i >> 6] >> (i &
63)) & 1ULL; }
};

```

## Implementation notes.

- **Kirsch–Mitzenmacher double hashing:** generating  $k$  hashes as  $h_1 + i \cdot h_2$  is provably as good asymptotically as  $k$  independent functions, and costs two hash evaluations instead of  $k$ . Standard practice.
  - The optimal parameters, derived from  $p_k = (1 - e^{-km/n})^k$ :  $k^* = (n/m) \cdot \ln 2$  and  $n = -m \cdot \ln p / (\ln 2)^2$ . At the optimum, roughly **half the bits are set**.
  - **Rule of thumb:** ~10 bits per item gives ~1% false positives with  $k = 7$ .
  - **Real uses:** Chrome's malicious-URL list, Bitcoin SPV, Cassandra/HBase/LevelDB SSTable lookups (skip a disk read if the key is definitely absent), spell checkers, network routers.
- 

## 8. Perfect Hashing

[Skiena §6.5, p.184]

### The goal

*The  $\Theta(n)$  worst-case search time for hashing is an annoyance, no matter how rare it is. Is there a way we can guarantee worst-case constant time search?*

*Perfect hashing offers this for **static** dictionaries. Here we are given all possible keys in one batch, and are not allowed to later insert or delete. This is a fairly common use case, so **why pay for the flexibility of dynamic data structures when you don't need them?***

### Why one level doesn't work — the birthday paradox

For a single collision-free table of size  $m$  holding  $n$  keys:

$$\Pr\{\text{no collision}\} = \prod_{i=0}^{n-1} (m-i)/m = m! / (m^n (m-n)!)$$

With  $m = 365$  this is the birthday problem: the probability drops below  $1/2$  at  $n = 23$  and below 3% at  $n \geq 50$ .

*Solving this asymptotically, we begin to expect collisions when  $n = \Theta(\sqrt{m})$ , or equivalently when  $m = \Theta(n^2)$ .*

But quadratic space seems like an awfully large penalty to pay for constant time access to  $n$  elements.

## The two-level (FKS) scheme

1. Hash the  $n$  keys into a **first-level table with  $n$  slots**. Collisions happen; let  $\ell_i$  be the length of bucket  $i$ .
2. **The condition:**  $N = \sum \ell_i^2 = \Theta(n)$ .
3. For bucket  $i$ , allocate  $\ell_i^2$  slots in a **second-level table** and hash its  $\ell_i$  items there. By the birthday-paradox threshold,  $\ell_i^2$  slots for  $\ell_i$  items means **no collision, with good probability** — and if there is one, just try another hash function for that bucket.

Why  $\sum \ell_i^2 = \Theta(n)$  is the right condition:

Suppose all elements were in lists of length  $\ell$ , meaning  $n/\ell$  non-empty lists. The sum of squares is  $N = (n/\ell) \ell^2 = n\ell$ , which is **linear because  $\ell$  is a constant**. We can even get away with a fixed number of lists of length  $\sqrt{n}$  and still use linear space.

In fact, it can be shown that  $N \leq 4n$  **with high probability**. So if this isn't true for the first hash function we try, we can just try another. Pretty soon we will find one with short-enough list lengths.

**Lookup.**  $h_1(s)$  gives the first-level slot, which stores the start/stop positions of this bucket's region in the second-level table **plus the identifier of the hash function for that region**. Then compute  $\text{start} + (h_2(s) \bmod (\text{stop} - \text{start}))$ .

Thus, search can always be performed in  $\Theta(1)$  time, using **linear storage** between the two tables.

Perfect hashing is a very useful data structure in practice, **ideal for whenever you will be making large numbers of queries to a static dictionary**. ... **minimum perfect hashing** guarantees constant-time access with **zero empty hash table slots**, resulting in an  $n$ -element second hash table for  $n$  keys.

**Engineering examples:** keyword recognition in compilers/lexers (`gperf`), static routing tables, CDN edge configs, read-only databases.

---

## 9. Rabin–Karp — hashing for string matching

[Skiena §6.7, p.188]

## The idea

Compare  $h(p)$  against the hash of each length- $m$  window of  $t$ .

*If two strings are identical, the resulting hash values must be the same. If different, the hash values will almost certainly be different. **These false positives should be so rare that we can easily spend the  $O(m)$  time it takes to explicitly check** whenever hash values agree.*

**The catch:** computing an  $m$ -character hash takes  $O(m)$ , and there are  $n - m + 1$  windows — back to  $O(nm)$ .

## The rolling hash — the entire trick

With  $H(S, j) = \sum_{i=0}^{m-1} \alpha^{m-(i+1)} \cdot \text{char}(s_{\{i+j\}})$ , the next window shares  $m-1$  characters:

$$H(S, j+1) = \alpha \cdot (H(S, j) - \alpha^{m-1} \cdot \text{char}(s_j)) + \text{char}(s_{\{j+m\}})$$

→ **C++ implementation:** [A5 The rolling hash \(Rabin–Karp\)](#)

*Once we know the hash value from the  $j$ -th position, we can find the hash value from the  $(j+1)$ -th for the cost of **two multiplications, one addition, and one subtraction**. ... This math works even if we compute  $H(S, j) \bmod M$ , where  $M$  is a reasonably large prime.*

**Total:**  $O(n + m)$  expected.

## The randomization

*Rabin–Karp is a good example of a **randomized algorithm** (if we pick  $M$  in some random way). We get **no guarantee** the algorithm runs in  $O(n+m)$  time, because we may get unlucky and have hash values frequently collide with spurious matches. Still, the odds are heavily in our favor — if the hash function returns values uniformly from  $0$  to  $M-1$ , **the probability of a false collision should be  $1/M$** . If  $M \approx n$ , there should only be one false collision per string; **if  $M \approx n^k$  for  $k \geq 2$ , the odds are great we will never see any**.*

**And the design note about the hash function** [Fig. 6.11]: a weak hash that merely *adds* character codes produces many collisions; the polynomial form gives distinctive codes to different substrings. **Position must matter.**

## C++ Implementation

```
#include <string>
#include <vector>
#include <cstdint>
#include <random>

// Rabin-Karp substring search. Expected  $O(n + m)$ ; worst case  $O(nm)$ 
// if the
// adversary knows the modulus (hence the randomized base).
vector<int> rabinKarp(const string& text, const string& pattern) {
    vector<int> matches;
    const int textLen = static_cast<int>(text.size());
    const int patLen = static_cast<int>(pattern.size());
    if (patLen == 0 || patLen > textLen) return matches;

    constexpr uint64_t kMod = (1ULL << 61) - 1;          //
    Mersenne prime
    static mt19937_64 rng(random_device{}());
    const uint64_t base = rng() % (kMod - 300) + 257;    //
    randomized base

    auto mulMod = [](uint64_t x, uint64_t y) {
        return static_cast<uint64_t>((static_cast<__uint128_t>(x) *
y) % kMod);
    };

    // highPow =  $\text{base}^{(m-1)} \bmod kMod$  -- the weight of the character
    leaving the window.
    uint64_t highPow = 1;
    for (int i = 0; i < patLen - 1; ++i) highPow = mulMod(highPow,
base);

    uint64_t hPat = 0, hWin = 0;
    for (int i = 0; i < patLen; ++i) {
        hPat = (mulMod(hPat, base) + static_cast<unsigned char>
(pattern[i])) % kMod;
```

```

        hWin = (mulMod(hWin, base) + static_cast<unsigned char>
(text[i])) % kMod;
    }

    for (int start = 0; ; ++start) {
        // Hash match is necessary but not sufficient -- verify in
O(m).
        if (hWin == hPat && text.compare(start, patLen, pattern) ==
0) matches.push_back(start);
        if (start + patLen >= textLen) break;
        // Roll: drop text[j], shift, add text[j+m].
        const uint64_t out =
            mulMod(highPow, static_cast<unsigned char>
(text[start]));
        hWin = (hWin + kMod - out) % kMod;           //
+kMod avoids underflow
        hWin = (mulMod(hWin, base) + static_cast<unsigned char>
(text[start + patLen])) % kMod;
    }
    return matches;
}

```

### Implementation notes.

- **Always verify on a hash match.** Skipping verification turns a Las Vegas algorithm into a Monte Carlo one with a silent error.
- **+ kMod before subtracting** — unsigned underflow otherwise.
- **Randomize the base** per run. With a fixed base and modulus, an adversary can construct  $\Theta(nm)$  inputs — this is a real attack on competitive-programming submissions ("anti-hash tests").
- $2^{61} - 1$  with `__int128` products is the standard safe choice.
- **Rabin–Karp's real strength is multi-pattern search:** put `k` pattern hashes in a set and find all of them in one pass — something KMP cannot do without Aho–Corasick.

**Comparison** (full treatment in [M18 \(planned\)](#)):

| ALGORITHM         | TIME                          | NOTES                                 |
|-------------------|-------------------------------|---------------------------------------|
| Naive             | $\theta(nm)$ worst            | simple                                |
| <b>Rabin–Karp</b> | $\theta(n+m)$ <b>expected</b> | multi-pattern; 2-D matching           |
| KMP               | $\theta(n+m)$ <b>worst</b>    | deterministic; needs failure function |
| Boyer–Moore       | sublinear typical             | best in practice for long patterns    |

## 10. Minwise Hashing

[Skiena §6.6, p.186]

### The problem

**Jaccard similarity** of two documents' vocabulary sets:

$$J(D_1, D_2) = |D_1 \cap D_2| / |D_1 \cup D_2|$$

Computing it exactly needs a hash table over all words of both documents. **What if you're allowed to look at only one word per document?**

*A first thought might be to pick the most frequent word, but it is likely to be "the" and tells you very little. Picking the most representative word, perhaps by TF-IDF, would be better. But it still makes assumptions about the distribution of words.*

### The idea

*The key idea is to **synchronize**, so we pick the same word out of the two documents while looking at the documents separately.*

Compute  $h(w_i)$  for every word and keep **the word with the smallest hash code**. Do the same for  $D_2$  **with the same hash function**.

*If the vocabularies were identical, the word with minimum hash code will be the same in both, and we get a match. In contrast, had we picked completely random words, the probability of matching would be only  $1/v$ .*

**The theorem:**

The probability that the minhash word appears in both documents ... is **exactly the Jaccard similarity**.

(Intuition: over the union  $D_1 \cup D_2$ , every element is equally likely to be the global minimum. The minhashes agree exactly when that minimum lies in the intersection — probability  $|D_1 \cap D_2| / |D_1 \cup D_2|$ .)

Sampling the  $k$  smallest hash values and reporting  $|intersection| / k$  gives a better estimate.

## Why bother — the indexing payoff

The alert reader may wonder why we bother. It takes time linear in the size of  $D_1$  and  $D_2$  to compute all the hash values just to find the minhash values, **yet this is the same running time it would take to compute the exact intersection using hashing!**

The value of minhash comes in building indexes for similarity search and clustering over large corpora. Hashing all words in all  $N$  documents gives a table of size  $O(Nm)$ . Storing  $k \ll m$  minwise hash values from each document will be much smaller at  $O(Nk)$ , but the documents at the intersection of the buckets associated with the  $k$  minwise hashes of a query  $Q$  are likely to contain the most similar documents.

**This is the foundation of locality-sensitive hashing (LSH)** — near-duplicate web page detection, recommender systems, plagiarism detection at scale.

## Stop and Think: estimating stream cardinality in $O(1)$ space

**Problem.** A stream of  $n$  numbers with many duplicates. Estimate the number of **distinct** values using constant memory.

**Solution.** Hash each element and keep only the **running minimum**  $h(s)$ .

If  $k$  values are drawn uniformly from  $[0, M-1]$ , what is the expected minimum? For  $k = 1$  it is obviously  $M/2$ . In general, hand-waving that  $k$  equally spaced values put the minimum at  $M/(k+1)$  gives the right answer:

$$P(X = i) = ((M-i)/M)^k - ((M-i-1)/M)^k \quad \Rightarrow \quad E[X] \rightarrow M/(k+1)$$

*The punch line is that  $m$  divided by the minhash value gives an excellent estimate of the number of distinct values. This method will not be fooled by repeated values in the stream, since repeated occurrences will yield precisely the same value every time.*

That duplicate-immunity is the essential property, and it is the seed of **HyperLogLog** — production cardinality estimation in Redis, BigQuery, and Presto, giving ~2% error in ~12 KB regardless of stream size.

## Chapter in One Page

| CONCEPT                     | THE ONE-LINE VERSION                                                                              |
|-----------------------------|---------------------------------------------------------------------------------------------------|
| Direct addressing           | One slot per possible key. $O(1)$ worst case, $\Theta( U )$ space.                                |
| Hash table                  | $\Theta( K )$ space, $O(1)$ <b>average</b> — the trade is worst case for space.                   |
| Collision                   | Unavoidable: $ U  > m$ by pigeonhole.                                                             |
| Independent uniform hashing | The ideal (a "random oracle"): uniform + independent. Not implementable.                          |
| Load factor                 | $\alpha = n/m$ . Everything is expressed in it.                                                   |
| Chaining                    | Chain per slot. INSERT $O(1)$ ; DELETE $O(1)$ with doubly linked lists.                           |
| Theorems 11.1 & 11.2        | Both unsuccessful <b>and</b> successful chained search are $\Theta(1 + \alpha)$ .                 |
| Hash tables destroy order   | Successor/Min/Max are $\Theta(n + m)$ — must scan all buckets.                                    |
| Division method             | $h(k) = k \bmod m$ ; $m$ prime, not near a power of 2. Anagram-collides at $m = 2^p - 1$ .        |
| Multiply-shift              | $h_a(k) = (ka \bmod 2^w) \gg (w - \ell)$ . Three instructions. <b><math>a</math> must be odd.</b> |
| The adversary               | Any <b>fixed</b> $h$ has a key set forcing $\Theta(n)$ . Randomize the function.                  |
| <b>Universal family</b>     | $\Pr\{h(k_1) = h(k_2)\} \leq 1/m$ for distinct keys.                                              |
| Theorem 11.4                | $h_{\{ab\}}(k) = ((ak+b) \bmod p) \bmod m$ is universal. $m$ need not be prime.                   |
| Theorem 11.5                | Odd- $a$ multiply-shift is $2/m$ -universal. Use this.                                            |
| Corollary 11.3              | Universal + chaining $\Rightarrow \Theta(s)$ expected for any $s$ operations. <b>No</b>           |

## bad input exists.

|                              |                                                                                                                           |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| d-independence               | $\Pr\{h(k_i)=q_i \ \forall i\} = 1/m^d$ . Linear probing needs $d = 5$ .                                                  |
| Open addressing              | Everything in the table; no pointers; $\alpha \leq 1$ ; probe sequence must be a permutation.                             |
| Deletion                     | Needs <b>tombstones</b> in general — which decouple search time from $\alpha$ .                                           |
| Linear probing               | $h(k, i) = (h_1(k) + i) \bmod m$ . Only $m$ sequences. Deletion <b>without</b> tombstones is possible.                    |
| Double hashing               | $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$ . $h_2(k)$ must be <b>coprime to m</b> . $\Theta(m^2)$ sequences.           |
| Theorem 11.6                 | Unsuccessful search $\leq 1/(1-\alpha)$ probes. $\alpha=0.5 \rightarrow 2$ ; $\alpha=0.9 \rightarrow 10$ .                |
| Theorem 11.8                 | Successful search $\leq (1/\alpha) \ln(1/(1-\alpha))$ . $\alpha=0.5 \rightarrow 1.387$ ; $\alpha=0.9 \rightarrow 2.559$ . |
| Primary clustering           | Bad in the RAM model, <b>good</b> under a memory hierarchy (same cache block).                                            |
| Theorem 11.9                 | 5-independent $h_1 + \alpha \leq 2/3 \Rightarrow$ linear probing is expected $O(1)$ .                                     |
| Modern design                | <b>Expensive hash function (in registers) + linear probing + low load factor.</b>                                         |
| Max load                     | $\Theta(\lg n / \lg \lg n)$ — hash tables are <b>not</b> $O(1)$ at the tail.                                              |
| Duplicate detection          | Hash the document; compare hash codes; verify only on collision.                                                          |
| Plagiarism                   | Hash <b>all overlapping windows</b> of length $w$ . Table becomes corpus-sized $\rightarrow$ minhash.                     |
| Commitment                   | Publish $h(\text{bid})$ before the deadline, the bid after.                                                               |
| <b>Canonicalization</b>      | Deliberately make equivalent objects collide (sorted letters, Soundex, stemming).                                         |
| <b>Fingerprinting</b>        | Represent big objects by small codes; sort/compare those, verify on collision.                                            |
| Bloom filter                 | $k$ hashes, $k$ bits per key. <b>No false negatives</b> . $(km/n)^k$ false positives.                                     |
| Bloom sizing                 | $k^* = (n/m) \ln 2$ ; $\sim 10$ bits/item $\rightarrow \sim 1\%$ error with $k = 7$ .                                     |
| Birthday paradox             | Zero collisions needs $m = \Theta(n^2)$ — too much space for one level.                                                   |
| <b>Perfect hashing (FKS)</b> | Two levels: $n$ buckets, then $\ell_i^2$ slots each. Works because $\sum \ell_i^2 \leq 4n$ whp. Static only.              |
| Rabin–Karp                   | Rolling hash: $H(j+1) = \alpha(H(j) - \alpha^{m-1}c_j) + c_{j+m}$ . $\Theta(n+m)$ expected. <b>Always verify.</b>         |

|                 |                                                                                                              |
|-----------------|--------------------------------------------------------------------------------------------------------------|
| Minwise hashing | $\Pr\{\text{minhashes agree}\} = \text{Jaccard similarity}$ . $k$ minhashes index $N$ documents in $O(Nk)$ . |
| Distinct-count  | Keep the running min hash; $M / \text{minhash} \approx$ number of distinct values.                           |
| The inversion   | Collisions are a <b>bug</b> in a dictionary and a <b>feature</b> in a fingerprint.                           |

---

## Recognition Table

| CLUE                                                                             | TECHNIQUE                                                           |
|----------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Membership / lookup by exact key, order irrelevant                               | hash table                                                          |
| Need <code>Successor</code> , <code>Min</code> , range queries, sorted iteration | <b>not</b> a hash table — balanced BST ( <a href="#">M08</a> )      |
| Adversary controls the keys (user input, network)                                | <b>randomize the hash function per process</b>                      |
| Static key set, need worst-case $O(1)$                                           | perfect hashing (FKS / <code>gperf</code> )                         |
| "Have I seen this before?", false positives OK, memory tight                     | <b>Bloom filter</b>                                                 |
| Avoid an expensive disk/network lookup for absent keys                           | Bloom filter as a pre-filter                                        |
| Substring search, especially <b>many patterns at once</b>                        | Rabin–Karp rolling hash                                             |
| Near-duplicate detection at web scale                                            | minwise hashing / LSH                                               |
| Count distinct elements in a stream, $O(1)$ memory                               | minhash → HyperLogLog                                               |
| Group anagrams / spelling variants / equivalent forms                            | <b>canonicalization</b> — hash the canonical form                   |
| Compare huge objects cheaply                                                     | <b>fingerprinting</b> — hash, compare codes, verify                 |
| Prove a file hasn't changed                                                      | cryptographic hash (commitment)                                     |
| Deletions are frequent                                                           | <b>chaining</b> (tombstones break open addressing's $\alpha$ bound) |
| Cache performance matters, few deletions                                         | <b>linear probing</b> , $\alpha \leq 0.6$ , strong hash             |
| Table size is a power of two                                                     | mask instead of <code>%</code> — but only with a strong hash        |
| p99 latency matters                                                              | remember max load is $\Theta(\log n / \log \log n)$                 |
| Sum of squares of bucket sizes is linear                                         | perfect hashing is affordable                                       |

## Common Mistakes Recap

1. Claiming hash tables are worst-case  $O(1)$ . They are  $O(1)$  **expected**,  $\Theta(n)$  worst case,  $\Theta(\log n / \log \log n)$  at the tail.

2. Using a **fixed** hash function on adversary-controlled keys → hash-flooding DoS.
  3. Deleting under open addressing by writing `NIL` instead of a tombstone.
  4. Double hashing where  $h_2(k)$  shares a factor with  $m$  (or returns 0).
  5.  $m = 2^p - 1$  with the division method on string keys — all anagrams collide.
  6. Power-of-two masking with a weak hash function.
  7. Forgetting to **verify** a Rabin–Karp hash match.
  8. Unsigned underflow in the rolling-hash subtraction ( $+ \bmod$  first).
  9. Fixed base/modulus in Rabin–Karp → anti-hash test attacks.
  10. Believing a Bloom filter can have false **negatives**, or trying to delete from one.
  11. Adding hash functions to a Bloom filter without checking the load — past the crossover it makes things worse.
  12. Expecting single-level perfect hashing in  $O(n)$  space (birthday paradox needs  $\Theta(n^2)$ ).
  13. Using a hash table when you need order, then paying  $\Theta(n + m)$  per `Successor`.
  14. Running open addressing near  $\alpha = 1$  — unsuccessful search goes to  $1/(1-\alpha)$ .
  15. Optimizing the number of probes while ignoring cache blocks.
- 

## Self-Test

1. Why does direct addressing fail, and what exactly does hashing trade for it? (§1)
2. Define independent uniform hashing. Which two properties do the chaining proofs actually need? (§1)
3. Prove Theorem 11.1. Then explain why Theorem 11.2's result is surprising. (§2)
4. Why is `Successor`  $\Theta(n + m)$  in a hash table? (§2)
5. Give the multiply-shift formula and explain each operation. Why must  $a$  be odd? (§3)
6. Give the pigeonhole argument that any fixed hash function has a bad key set. (§3)
7. Define universal and  $\epsilon$ -universal. Is every universal family uniform? (§3)
8. Prove  $h_{\{ab\}}(k) = ((ak+b) \bmod p) \bmod m$  is universal — all three steps. (§3)
9. Why does Corollary 11.3 say an adversary cannot force the worst case? (§3)
10. Why can't you delete from an open-address table by writing `NIL`? What does the fix cost? (§4)

11. Why must  $h_2(k)$  be coprime to  $m$ ? Give two ways to guarantee it. (§4)
  12. Derive  $1/(1-\alpha)$  and give the intuitive series interpretation. (§4)
  13. Give the probe counts at  $\alpha = 0.5$  and  $\alpha = 0.9$  for both search types. Why is the asymmetry a design lesson? (§4)
  14. What is primary clustering, and why is CLRS's verdict on it different in the two models? (§5)
  15. State Theorem 11.9 and the modern hash-table design it implies. (§5)
  16. Give Skiena's three duplicate-detection applications and why each needs a different scheme. (§6)
  17. What is canonicalization? Why are collisions *desirable* there? (§6)
  18. How does a Bloom filter get  $51\times$  better error than a hash table at the same memory? Which error is impossible? (§7)
  19. Why does increasing  $k$  eventually hurt? (§7)
  20. Why does one-level perfect hashing need  $\Theta(n^2)$  space, and how does FKS avoid it? (§8)
  21. Why is  $\sum \ell_i^2 = \Theta(n)$  the right condition, and why is it achievable? (§8)
  22. Give the Rabin–Karp rolling-hash update. Why must you still verify? (§9)
  23. Why randomize the base and modulus? (§9)
  24. Why is  $\Pr\{\text{minhashes agree}\}$  exactly the Jaccard similarity? (§10)
  25. Estimate the number of distinct values in a stream in  $O(1)$  memory. Why is it immune to duplicates? (§10)
- 

## Practice — where to drill this module

| IDEA IN THIS MODULE               | PROBLEM                                                                              | WHY IT'S THE RIGHT DRILL                                                                                   |
|-----------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| The hash map as a reflex          | <a href="#">1 · Two Sum</a>                                                          | $\Theta(n^2) \rightarrow \Theta(n)$ by remembering what you have seen; the smallest possible demonstration |
| Hashing a <i>derived</i> key      | <a href="#">49 · Group Anagrams</a>                                                  | the key is the sorted string (or the 26-count signature) — designing the key <b>is</b> the problem         |
| Set membership beats sorting      | <a href="#">128 · Longest Consecutive Sequence</a>                                   | $\Theta(n)$ with a hash set vs $\Theta(n \lg n)$ sorted; and why the inner loop is still linear overall    |
| Build the table yourself          | <a href="#">706 · Design HashMap</a> · <a href="#">705 · Design HashSet</a>          | <a href="#">A2</a> and <a href="#">A3</a> below, as a submission                                           |
| Hash + list for $O(1)$ everything | <a href="#">146 · LRU Cache</a> · <a href="#">380 · Insert Delete GetRandom O(1)</a> | the map stores <i>iterators</i> , which is why erase is $O(1)$ — see <a href="#">M06</a>                   |
| Counting with a map               | <a href="#">347 · Top K Frequent Elements</a>                                        | frequency map then bucket sort — $\Theta(n)$ end to end                                                    |
| Rolling hash                      | <a href="#">187 · Repeated DNA Sequences</a>                                         | fixed-width windows, so the rolling hash is at its simplest                                                |
| Rolling hash, seriously           | <a href="#">1044 · Longest Duplicate Substring</a>                                   | binary search on length + Rabin–Karp; <b>the</b> hard hashing problem                                      |
| Substring search                  | <a href="#">28 · Find the Index of the First Occurrence in a String</a>              | submit naive, then <a href="#">A5</a> below                                                                |

**Beyond LeetCode.** [CSES Problem Set](#) — *String Algorithms* (several are Rabin–Karp by design). [Codeforces](#) [hashing](#) tag · [strings](#) tag. **A warning worth heeding on Codeforces:** anti-hash tests against fixed `unordered_map` seeds are a real and common hack — see toolkit §5.

## C++ Toolkit for This Module

Language material from Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., ch. 5 and §1.6.4.

## 1. `unordered_map` / `unordered_set` — separate chaining, in the standard

Both `libstdc++` and `libc++` implement `unordered_map` as **separate chaining** (A2), not open addressing. That is mandated in spirit by the standard's guarantees: references to elements must stay valid across rehashing, which open addressing cannot provide.

| OPERATION                                | UNORDERED_MAP                                               | MAP (RED-BLACK TREE, M08)              |
|------------------------------------------|-------------------------------------------------------------|----------------------------------------|
| find / insert / erase                    | <code>O(1)</code> <b>expected</b> , <code>O(n)</code> worst | <code>O(lg n)</code> <b>guaranteed</b> |
| ordered iteration                        | no                                                          | yes                                    |
| <code>lower_bound</code> / range queries | no                                                          | yes                                    |
| memory per element                       | higher (bucket array + node)                                | node only                              |
| adversarial input                        | <b>can be attacked</b>                                      | safe                                   |

## 2. `std::hash`, and how to hash your own type

`unordered_map<K, V>` needs `std::hash<K>`. It is specialised for the built-ins and `string`, but not for your struct or for `pair`:

```
struct Point { int x, y; bool operator==(const Point& o) const {
    return x == o.x && y == o.y; } };

// Option A: specialise std::hash. Note this MUST go in namespace
std -- one of
// the very few places the standard permits you to add to it.
namespace std {
template <> struct hash<Point> {
    size_t operator()(const Point& point) const noexcept {
        // Do NOT write hash<int>{}(p.x) ^ hash<int>{}(p.y): XOR is
        symmetric,
        // so (3,5) and (5,3) collide, and (a,a) hashes to 0 for
        every a.
        size_t combined = hash<int>{}(point.x);
        combined ^= hash<int>{}(point.y) + 0x9e3779b97f4a7c15ULL +
        (combined << 6) + (combined >> 2);
    }
};
```

```

        return combined;    // boost::hash_combine's mixer: the
        constant is 2^64/phi
    }
};

// Option B: a function object passed as the third template
// argument -- Weiss's
// function-object idiom [1.6.4, p.41] applied to hashing.
struct PointHash {
    size_t operator()(const Point& point) const noexcept {
        return (size_t)point.x * 1000003u + (size_t)point.y;
    }
};

using PointSet = unordered_set<Point, PointHash>;

```

The hash must be consistent with `operator==`: equal keys must hash equal. That is the entire contract, and violating it produces a container that silently loses elements.

### 3. `reserve` and the load factor

```

void reserveDemo(int n) {
    unordered_map<int,int> table;
    table.reserve(n);           // pre-size the bucket array:
    no rehashing during fill
    table.max_load_factor(0.5f); // trade memory for fewer
    collisions (default is 1.0)
}

```

`reserve(n)` is worth a factor of  $\sim 2$  on a large fill, because it removes the  $O(\lg n)$  rehashes — each of which is a full  $\Theta(n)$  rebuild (M09 explains why the total is still amortized  $O(1)$  per insert).

### 4. Integer division and `%` are expensive

$h(k) = k \bmod m$  costs a hardware divide: **20–40 cycles**, versus 3 for a multiply. That is a large part of why the multiply-shift method of A4 exists — it is a multiply and a shift, and when  $m = 2^\ell$  the modulo disappears entirely. Note  $k \% m$  with a *negative*  $k$  is negative in C++, which is why every hash below casts to `unsigned long long` first.

## 5. Hash flooding — the attack the theory predicts

`unordered_map`'s  $\mathcal{O}(n)$  worst case is not hypothetical. If an attacker can choose your keys and knows your hash function, they can drive every key into one bucket and turn an  $\mathcal{O}(1)$  lookup into  $\mathcal{O}(n)$ , collapsing a web service. This was a real CVE class across PHP, Python, Ruby and Java in 2011.

**The fix is exactly Theorem 11.4:** choose the hash *randomly at run time* from a universal family, so the adversary cannot know it. In competitive programming the same trick defeats anti-hash tests:

```
struct SafeHash {
    size_t operator()(uint64_t key) const {
        // A per-process random offset the adversary cannot
        predict.

        static const uint64_t FIXED =
            chrono::steady_clock::now().time_since_epoch().count();
        key += FIXED + 0x9e3779b97f4a7c15ULL;
        key = (key ^ (key >> 30)) * 0xbf58476d1ce4e5b9ULL;    //
        splitmix64
        key = (key ^ (key >> 27)) * 0x94d049bb133111ebULL;
        return (size_t)(key ^ (key >> 31));
    }
};
```

## 6. `unsigned __int128` for modular multiplication

`(a * k + b) % p` with 64-bit `a` and `k` overflows before the modulo. GCC/Clang's 128-bit integer type is the practical fix:

```
uint64_t mulmod(uint64_t multiplier, uint64_t key, uint64_t offset,
uint64_t prime) {
    return (uint64_t)((((unsigned __int128)multiplier * key +
offset) % prime));
}
```

It is a compiler extension, not standard C++, but it is available on every judge and on both major compilers.

## 7. `vector<bool>` — the specialisation that is not a container

`vector<bool>` is bit-packed: it stores 8 booleans per byte, `operator[]` returns a **proxy object** rather than a `bool&`, and you cannot take `&v[i]`. That makes it wrong for general use (`vector<char>` is the fix) — but exactly right for a **Bloom filter**, where 8× memory density is the entire point. A6 uses it deliberately.

---

## Appendix — C++ for Every Pseudocode Block

```
// Shared prelude: a random source for the universal family and the
// Bloom filter.
mt19937_64& rng() { static mt19937_64 g(20260903u); return g; }
static uint64_t randU64() { return rng(); }
```

### A1 DIRECT-ADDRESS-SEARCH / INSERT / DELETE

*Pseudocode: §1, "Direct-address tables".*

```
// One slot per POSSIBLE key. Templated on the stored type, with
// the assumption
// stated as Weiss instructs [1.6.1, p.37]: T is any type; we store
// pointers to
// it, so T needs nothing at all.
template <typename T>
class DirectAddressTable {
public:
    // `explicit` prevents `DirectAddressTable<int> t = 1000;` from
    // compiling --
    // a size is not a table [Weiss 1.4.2].
    explicit DirectAddressTable(size_t universeSize) :
    slot_(universeSize, nullptr) {}

    T* search(size_t key) const { return slot_[key]; } //
    DIRECT-ADDRESS-SEARCH
    void insert(size_t key, T* item) { slot_[key] = item; }
    // DIRECT-ADDRESS-INSERT
    void erase(size_t key) { slot_[key] = nullptr; } //
    DIRECT-ADDRESS-DELETE

    size_t universeSize() const { return slot_.size(); }
private:
```

```

    // Pointers, not values: NIL has to be representable, and
    nullptr is it.
    // Storing T by value would need a separate "occupied" flag.
    vector<T*> slot_;
};

```

**Complexity.**  $\Theta(1)$  worst case for all three — no hashing, no collisions, nothing to analyse.

**And that is why the chapter exists.** Space is  $\Theta(|U|)$ , the size of the *key universe*, not the number of keys. For 64-bit keys that is  $1.8 \times 10^{19}$  slots to store ten items. Direct addressing is the right answer exactly when the universe is small and dense — day-of-year, HTTP status code, ASCII byte — and hopeless otherwise.

*Measured: a 1000-slot table holding 3 keys. That ratio **is** the problem it has.*

## A2 CHAINED-HASH-INSERT / SEARCH / DELETE

*Pseudocode: §2, "The structure".*

```

class ChainedHashTable {
public:
    explicit ChainedHashTable(size_t slotCount) : table_(slotCount)
    {}

    // The cast to unsigned matters: in C++, (-7) % 97 is -7, and a
    // negative
    // index into a vector is undefined behaviour. Cast FIRST, then
    // take mod.
    size_t hash(long long key) const {
        return (size_t)((((unsigned long long)key) % table_.size()));
    }

    // CHAINED-HASH-INSERT(T, x): LIST-PREPEND(T[h(x.key)], x)
    //
    // Note what the pseudocode does NOT do: it does not check
    // whether the key
    // is already present. Insertion is  $O(1)$  precisely BECAUSE it
    // skips that
    // check, and the table therefore behaves like a MULTISSET. CLRS
    // is explicit:

```

```

    // if you need set semantics, search first, and insertion
    becomes O(1+alpha).
    void insert(long long key) { table_[hash(key)].push_front(key);
}

    // CHAINED-HASH-SEARCH(T, k): return LIST-SEARCH(T[h(k)], k)
    bool search(long long key) const {
        // `const list<...>&`, not a copy: binding a reference to
    the bucket is
        // Weiss's "aliasing a complicated name" use of references
    [1.5.2, p.24].
        const list<long long>& chain = table_[hash(key)];
        return find(chain.begin(), chain.end(), key) !=
chain.end();
    }

    // CHAINED-HASH-DELETE(T, x): LIST-DELETE(T[h(x.key)], x)
    // The pseudocode is given a POINTER to x, so it is O(1).
    Deleting by VALUE,
    // as here, must search the chain first -- O(1 + alpha).
    bool erase(long long key) {
        list<long long>& chain = table_[hash(key)];
        auto it = find(chain.begin(), chain.end(), key);
        if (it == chain.end()) return false;
        chain.erase(it);          // std::list::erase is O(1) given
    the iterator
        return true;
    }

    size_t longestChain() const {
        size_t mx = 0;
        for (const auto& chain : table_) mx = max(mx,
chain.size());
        return mx;
    }
private:
    vector<list<long long>> table_;
};

```

**Complexity.** Let  $\alpha = n/m$  be the **load factor** — the average chain length.

- **INSERT:**  $O(1)$  worst case (prepend, no search).
- **SEARCH:**  $\Theta(1 + \alpha)$  **average**, under simple uniform hashing;  $\Theta(n)$  worst case (everything in one chain).
- **DELETE:**  $O(1)$  given a pointer,  $\Theta(1 + \alpha)$  by key.

$\alpha = O(1)$  makes all operations  $O(1)$  on average, which is the whole design: keep  $m$  proportional to  $n$  by rehashing, and the average chain never grows.

Measured,  $m = 1000$  slots, random keys:

| A   | LONGEST CHAIN | MEAN NON-EMPTY CHAIN |
|-----|---------------|----------------------|
| 0.5 | 5             | 1.28                 |
| 1.0 | 6             | 1.58                 |
| 2.0 | 8             | 2.19                 |
| 4.0 | 12            | 4.03                 |
| 8.0 | 19            | 8.00                 |

The mean tracks  $\alpha$  exactly, as the theory says — but note the **longest** chain is roughly  $\alpha + \Theta(\lg n / \lg \lg n)$  even when the hash is perfect. Average-case  $O(1)$  never means "every bucket is small".

## A3 HASH-INSERT / HASH-SEARCH (open addressing)

Pseudocode: §5, "The structure".

```
class OpenAddressingTable {
public:
    // `enum class`, not a bare enum: the enumerators are SCOPED
    // (Probe::Linear, not just Linear) and do not implicitly
    convert to int.
    enum class Probe { Linear, Quadratic, Double };

    OpenAddressingTable(size_t slotCount, Probe strategy) :
    slot_(slotCount, EMPTY), probe_(strategy) {}

    // Two sentinel values, and the difference between them is the
    whole reason
    // deletion is hard in an open-addressed table:
    //  EMPTY  -- never used; a search may STOP here.
```

```

    // DELETED -- used and then vacated; a search must CONTINUE
through it,
    // or it would miss keys that probed past this
slot.
    // Marking deletions instead of clearing them is why search
time in an
    // open-addressed table degrades with the number of DELETIONS,
not just the
    // number of live keys. It is also why CLRS says: if you need
deletion,
    // prefer chaining.
static constexpr long long EMPTY    = LLONG_MIN;
static constexpr long long DELETED = LLONG_MIN + 1;

size_t primaryHash(long long key) const { return (size_t)
(((unsigned long long)key) % slot_.size()); }
size_t stepHash(long long key) const { return 1 + (size_t)
(((unsigned long long)key) % (slot_.size() - 1)); }
// ^^^ the +1 is essential: a step
of 0 would
// probe the same slot forever.

size_t probeAt(long long key, size_t probe) const {
    const size_t slotCount = slot_.size();
    switch (probe_) {
        case Probe::Linear:    return (primaryHash(key) +
probe) % slotCount;           // h1 + i
        case Probe::Quadratic: return (primaryHash(key) + probe
+ 3 * probe * probe) % slotCount; // h1 + c1 i + c2 i^2
        case Probe::Double:   return (primaryHash(key) + probe
* stepHash(key)) % slotCount; // h1 + i*h2
    }
    return 0;
}

long long probesLastOp = 0; // instrumentation

// HASH-INSERT(T, k)
bool insert(long long key) {
    const size_t slotCount = slot_.size();
    probesLastOp = 0;

```

```

        for (size_t probe = 0; probe < slotCount; ++probe) {
            // 2  repeat ... 8  until i == m
            ++probesLastOp;
            size_t slot = probeAt(key, probe);
        // 3  q = h(k, i)
            if (slot_[slot] == EMPTY || slot_[slot] == DELETED) {
slot_[slot] = key; return true; } // 4-5
            if (slot_[slot] == key) return true; //
already present
        }
        return false; // 9
    "hash table overflow"
    }

    // HASH-SEARCH(T, k)
    bool search(long long key) {
        const size_t slotCount = slot_.size();
        probesLastOp = 0;
        for (size_t probe = 0; probe < slotCount; ++probe) {
            ++probesLastOp;
            size_t slot = probeAt(key, probe);
            if (slot_[slot] == key) return true; // 4
        if T[q] == k: return q
            if (slot_[slot] == EMPTY) return false; // 7
        until T[q] == NIL
            // NOTE: DELETED does NOT stop the scan. That is the
        whole point.
        }
        return false; // 8
    return NIL
    }

    bool erase(long long key) {
        const size_t slotCount = slot_.size();
        for (size_t probe = 0; probe < slotCount; ++probe) {
            size_t slot = probeAt(key, probe);
            if (slot_[slot] == key) { slot_[slot] = DELETED; return
true; } // tombstone
            if (slot_[slot] == EMPTY) return false;
        }
        return false;
    }
}

```

```
private:
    vector<long long> slot_;    // no chains, no pointers: ONE
    contiguous array
    Probe probe_;
};
```

**Complexity.** With uniform hashing and  $\alpha < 1$ :

- **Unsuccessful search / insert:** at most  $1/(1-\alpha)$  probes.
- **Successful search:** at most  $(1/\alpha) \cdot \ln(1/(1-\alpha))$  probes.

Both **blow up** as  $\alpha \rightarrow 1$ :  $\alpha = 0.5$  costs 2 probes,  $\alpha = 0.9$  costs 10,  $\alpha = 0.99$  costs 100.

**Why open addressing exists:** no pointers, no allocations, one contiguous array — vastly better cache behaviour than chaining. **Why it is harder:** deletion needs tombstones, and it cannot exceed  $\alpha = 1$ .

Measured,  $m = 4096$ , average probes for an **unsuccessful** search:

| A    | $1/(1-A)$ BOUND | LINEAR        | QUADRATIC    | DOUBLE      |
|------|-----------------|---------------|--------------|-------------|
| 0.25 | 1.33            | 1.41          | 1.37         | <b>1.34</b> |
| 0.50 | 2.00            | 2.48          | <b>2.16</b>  | 2.62        |
| 0.75 | 4.00            | 7.08          | <b>4.40</b>  | 6.04        |
| 0.90 | 10.00           | 35.77         | <b>10.59</b> | 16.90       |
| 0.95 | 20.00           | <b>115.69</b> | <b>21.37</b> | 32.55       |

*This table is primary clustering, measured. The  $1/(1-\alpha)$  bound assumes uniform hashing — every probe sequence equally likely — which linear probing badly violates: occupied runs merge and grow, so a long run gets longer. At  $\alpha = 0.95$  linear probing costs **115.69 probes against a bound of 20**, while quadratic probing stays at 21.37. If you take one thing from this module's implementation advice: **do not use linear probing above  $\alpha \approx 0.5$** .*

## A4 The hash functions

Corresponds to the  $H(S)$ ,  $h(k) = k \bmod m$ ,  $h_a(k)$ , and  $h_{\{ab\}}(k)$  formula blocks in §3–4.

```
// DIVISION METHOD:  $h(k) = k \bmod m$ . Fast (one divide) and fragile.
```

```

size_t divisionHash(long long key, size_t slotCount) {
    return (size_t)((((unsigned long long)key) % slotCount));
}

// MULTIPLY-SHIFT, CLRS eq. (11.2):  $h_a(k) = (k \cdot a \bmod 2^w) \gg (w - 1)$ ,  $m = 2^l$ .
//
// `k * a` on uint64_t wraps modulo  $2^{64}$  -- and unlike SIGNED
// overflow, that
// wraparound is DEFINED behaviour, which is exactly the "mod  $2^w$ "
// the formula
// wants. The  $\gg$  then keeps the  $l$  MOST SIGNIFICANT bits of the low
// word, which
// mix contributions from ALL bits of  $k$ . That is why it survives
// the adversarial
// input below and the division method does not.
//
// `a` must be ODD (so the map is a bijection mod  $2^{64}$ ); this
// constant is
//  $2^{64} / \phi$ , the golden-ratio multiplier Knuth recommends.
size_t multiplyShiftHash(uint64_t key, unsigned tableBits) {
    static const uint64_t multiplier = 0x9E3779B97F4A7C15ULL |
1ULL;
    return (size_t)((key * multiplier) >> (64 - tableBits));
}

// THEOREM 11.4's universal family:  $h_{\{ab\}}(k) = ((a \cdot k + b) \bmod p) \bmod m$ .
//  $a$  and  $b$  are drawn ONCE AT CONSTRUCTION -- the randomness is in
// choosing the
// FUNCTION, not in hashing each key. Two calls with the same  $k$ 
// give the same
// slot; two different UniversalHash objects generally do not.
struct UniversalHash {
    uint64_t multiplier, offset, prime;
    size_t slotCount;
    UniversalHash(size_t slots, uint64_t primeModulus =
2147483647ULL) : prime(primeModulus), slotCount(slots) {    //  $p =$ 
231 - 1, prime
        multiplier = uniform_int_distribution<uint64_t>(1, prime -
1)(rng());    //  $a$  in  $\mathbb{Z}_p^*$  (never 0)

```

```

        offset = uniform_int_distribution<uint64_t>(0, prime - 1)
(rng());    // b in Z_p
    }
    size_t operator()(uint64_t key) const {
        // unsigned __int128 (toolkit 6): a*k overflows 64 bits
before the mod.
        return (size_t)((((unsigned __int128)multiplicator * key +
offset) % prime % slotCount));
    }
};

// Skiena's string hash: treat characters as digits in base alpha,
reduce mod m.
// Horner's rule -- one multiply-add per character, and the mod at
every step
// keeps h bounded so it never overflows.
uint64_t stringHash(const string& text, uint64_t base, uint64_t
modulus) {
    uint64_t hash = 0;
    // `unsigned char`, not `char`: plain char is SIGNED on x86, so
a byte >= 128
    // would contribute a negative value and wreck the hash for
non-ASCII input.
    for (unsigned char ch : text) hash = (hash * base + ch) %
modulus;
    return hash;
}

```

Measured. 100 000 **adversarial** keys — all multiples of 64 — into  $m = 1024$  slots:

| HASH                                 | SLOTS USED  | $\chi^2$  |
|--------------------------------------|-------------|-----------|
| division, $k \bmod 1024$             | 16 / 1024   | 6 300 000 |
| multiply-shift                       | 1024 / 1024 | 19.43     |
| universal $((ak+b) \bmod p) \bmod m$ | 1024 / 1024 | 32.10     |

The division method used **16 slots out of 1024** and left the table 98% empty with chains 6250 long. The reason is CLRS's warning made concrete: with  $m = 2^p$ ,  $h(k)$  is just the low  $p$  bits of  $k$  — and every key here has its low 6 bits zero. Both other methods spread the same keys perfectly (a  $\chi^2$  near the 1023 degrees of freedom would be expected for random data; these are far below, because the keys are evenly spaced).

And the anagram failure: with  $m = 2^8 - 1 = 255$ , `stringHash("abc") == stringHash("cab")` — every permutation of a string collides, exactly as Exercise 11.3-3 predicts. With a large prime modulus they differ.

## A5 The rolling hash (Rabin–Karp)

Corresponds to  $H(S, j+1) = \alpha \cdot (H(S, j) - \alpha^{m-1} \cdot \text{char}(s_j)) + \text{char}(s_{j+m})$  in §7.

```
// Returns every index where `pat` occurs in `text`.
vector<int> rabinKarp(const string& text, const string& pattern,
                    uint64_t base = 257, uint64_t modulus =
1000000007ULL,
                    long long* verifications = nullptr) {
    vector<int> matches;
    const int textLen = (int)text.size(), patLen =
(int)pattern.size();
    if (patLen == 0 || patLen > textLen) return matches;

    // alpha^(m-1) mod mod -- the weight of the character LEAVING
the window.
    uint64_t topWeight = 1;
    for (int i = 0; i < patLen - 1; ++i) topWeight = (topWeight *
base) % modulus;

    uint64_t patternHash = stringHash(pattern, base, modulus);
    uint64_t windowHash = stringHash(text.substr(0, patLen), base,
modulus);

    for (int start = 0; ; ++start) {
        if (windowHash == patternHash) {
            // A hash match is only EVIDENCE, never proof:
different strings can
            // share a hash. Verify with a real comparison.
Skipping this makes
            // the algorithm Monte Carlo (fast, occasionally wrong)
instead of
            // Las Vegas (always right, fast in expectation).
            if (verifications) ++*verifications;
            if (text.compare(start, patLen, pattern) == 0)
matches.push_back(start);
        }
    }
```

```

        if (start + patLen >= textLen) break;

        // THE ROLL, in two steps, and note the `+ mod` before
        subtracting:
        //   ht - x  on unsigned types WRAPS if x > ht. Adding
        `mod` first keeps
        //   the value positive; this is the single most common
        rolling-hash bug.

        windowHash = (windowHash + modulus - (topWeight * (unsigned
        char)text[start]) % modulus) % modulus;    // drop text[j]
        windowHash = (windowHash * base + (unsigned char)text[start
        + patLen]) % modulus;                      // shift, add text[j+m]
    }
    return matches;
}

```

**Complexity.**  $\Theta(n + m)$  **expected**;  $\Theta(nm)$  worst case if every window's hash collides with the pattern's (which a randomly chosen  $\alpha$  makes vanishingly unlikely — that is M04's lesson applied here).

**Why the roll is  $\mathcal{O}(1)$ :** consecutive windows share  $m - 1$  characters. Subtract the departing character's contribution, multiply by the base, add the arriving character — "two multiplications, one addition, and one subtraction", independent of  $m$ .

*Measured, on the input that is worst-case for **naive** matching ( $text = 200\,000$  as,  $pat = 99$  as then a b): Rabin–Karp performed **0 full string comparisons** across 199 901 windows and finished in 2 255  $\mu$ s. Naive matching would perform **19 990 100** character comparisons on the same input.*

## A6 The Bloom filter

Corresponds to  $p_k = (km/n)^k$  in §8.

```

class BloomFilter {
public:
    // bits = total bit array size, k = number of hash functions.
    BloomFilter(size_t bitCount, int hashCount) : bits_(bitCount,
    false), hashCount_(hashCount) {
        // k independent hashes, made by k random odd multipliers.
        Drawing them
    }
}

```

```

        // at run time is the universal-hashing idea again: an
adversary cannot
        // pre-compute a set of keys that all collide.
        for (int i = 0; i < hashCount; ++i)
seeds_.push_back(randU64() | 1ULL);
    }

    void insert(uint64_t key) { for (size_t bit : positions(key))
bits_[bit] = true; }

    // The name is the specification: a `true` MIGHT be wrong, a
`false` never is.
    bool mightContain(uint64_t key) const {
        for (size_t bit : positions(key)) if (!bits_[bit]) return
false;
        return true;
    }

    size_t bitsSet() const { return (size_t)count(bits_.begin(),
bits_.end(), true); }
    size_t size() const { return bits_.size(); }
private:
    // vector<bool> is the bit-packed specialisation (toolkit 7).
Everywhere else
    // it is a trap; here its 8x density is the entire reason the
structure exists.
    vector<bool> bits_;
    int hashCount_;
    vector<uint64_t> seeds_;

    vector<size_t> positions(uint64_t key) const {
        vector<size_t> positionsOut;
        positionsOut.reserve(hashCount_);
        for (uint64_t seed : seeds_) {
            uint64_t mixed = key * seed; //
multiply...
            mixed ^= mixed >> 29; mixed *= 0xBF58476D1CE4E5B9ULL;
mixed ^= mixed >> 32; // ...then AVALANCHE:
            // the shift-xor-multiply chain (splitmix64's
finaliser) makes every
            // input bit affect every output bit. Without it, `key
* s` leaves the

```

```

        // low bits barely mixed and the filter's error rate is
        far worse than
        // the formula predicts.
        positionsOut.push_back((size_t)(mixed % bits_.size()));
    }
    return positionsOut;
}
};

```

**Complexity.**  $O(k)$  per insert and per query — **independent of  $n$** . Space  $m$  bits total, typically 8–16 bits *per key* regardless of key size, which is the point: a Bloom filter for 10 million 100-byte URLs fits in 12 MB.

**The guarantee is one-sided. No false negatives, ever:** if a key was inserted, all  $k$  of its bits are set, so `mightContain` cannot return `false`. **False positives are possible** at rate  $\approx (1 - e^{-kn/m})^k$ , minimised at  $k = (m/n) \cdot \ln 2$ , where roughly **half the bits are set**.

Measured,  $n = 10\,000$  keys inserted, 100 000 absent keys queried:

| <i>BITS/KEY</i> | $k$ | <i>MEASURED FP RATE</i> | <i>PREDICTED</i> | <i>FILL</i> |
|-----------------|-----|-------------------------|------------------|-------------|
| 4               | 3   | 0.14451                 | 0.14689          | 52.7%       |
| 8               | 5   | 0.02161                 | 0.02168          | 46.7%       |
| 10              | 7   | 0.00769                 | 0.00819          | 50.3%       |
| 16              | 11  | <b>0.00047</b>          | 0.00046          | 49.8%       |

Measured and predicted agree to three decimal places, the fill sits at ~50% exactly as the optimality condition requires, and **no configuration ever produced a false negative**. 16 bits per key — 2 bytes — buys a 1-in-2000 error rate on membership queries of arbitrary-length keys.

---

Previous: [M06 — Elementary Data Structures](#) · Next: [M08 — Search Trees & Augmentation](#)